

RAPPORT FINAL

Optimisation conjointe déploiement réseau & stratégie énergétique en Afrique

18 octobre 2026

Labgoul Anas - Oujaa Haitam Yassine - Takfa Anass - Belfatmi Ayoub - Ait Mansour Abderrahmane

orange™

ÉCOLE POLYTECHNIQUE

IP PARIS

TABLE DES MATIÈRES

1 Introduction **1**
1.1 Contexte . . . 1
1.2 L'état de l'art . . . 1
1.3 Cadre et organisation . . . 2
1.4 Structure du manuscrit . . . 2

2 Formalisation du problème télécom **2**

3 Modèles mathématiques **4**
3.1 Modèle initial (MINLP^{initial}) . . . 4
3.2 Améliorations et renforcements (MILP^{initial}) . . . 6
3.2.1 Linéarisation de la contrainte de migration. . . . 6
3.2.2 Dépendance temporelle des paramètres de trafic. . . . 6
3.2.3 Monotonicité du déploiement des sites NG. . . . 6
3.2.4 Relaxation de l'intégrité des variables de population. . . . 7
3.2.5 MILP^{initial} . . . 7
3.2.6 Amélioration du choix du paramètre Big-M . . . 7
3.3 Reformulation (MILP^{reformulé}) . . . 8
3.3.1 Réduction de la contrainte (1.4) de décodage de l'indicatrice . . . 8
3.3.2 Linéarisation du nouveau modéle . . . 9
3.4 Comparaison des deux modèles . . . 11

4 Algorithmes heuristiques et métaheuristiques **12**
4.1 Motivations et positionnement . . . 12
4.2 Fix-and-Relax . . . 12
4.3 Algorithme génétique initial basé sur un solveur . . . 13
4.3.1 Principe de l'algorithme . . . 13
4.3.2 Complexité. . . . 15
4.3.3 Résultats et analyse . . . 16
4.3.4 Limites de cette approche. . . . 16
4.4 Algorithme génétique direct avec initialisation gloutonne structurée . . . 16
4.4.1 Nouveau codage des individus . . . 16
4.4.2 Évaluation directe de la fitness . . . 17
4.4.3 Initialisation gloutonne structurée . . . 19
4.4.4 Boucle génétique inchangée, mais plus efficace . . . 20
4.4.5 Complexité . . . 20
4.4.6 Résultats et analyse . . . 21
4.5 Algorithme mémétique direct . . . 21
4.5.1 Principe général . . . 22
4.5.2 Recherche locale *first-improvement* . . . 22
4.5.3 Mutations structurées . . . 23
4.5.4 Boucle mémétique . . . 24
4.5.5 Complexité . . . 24
4.5.6 Résultats et analyse . . . 25
4.5.7 Conclusion sur l'évolution des heuristiques. . . . 25

5 Mixed-Integer Linear Programming Graph Neural Network (MILPGNN) 26
5.1 Motivations et positionnement . . . 27
5.2 Représentation des données (Entrées et Sorties) . . . 27
5.3 Architecture du Graph Neural Network (MILPGNN) . . . 27
5.4 Formulation Mathématique de la Couche SAGEConv . . . 27
5.5 Résultats et analyse . . . 29

6 Apprentissage par Renforcement 30
6.1 Motivations et positionnement . . . 30
6.2 Formulation MDP . . . 31
6.3 Algorithme PPO et architecture . . . 31
6.4 Résultats et analyse . . . 32

7 Extension énergétique du problème 33
7.1 Motivations et positionnement . . . 33
7.2 Variables de décision et paramètres additionnels . . . 34
7.3 Modélisation de la consommation et de l'empreinte carbone . . . 34
7.4 Règles et conditions du retrait legacy . . . 35
7.5 Dynamique migratoire avec retrait et redistribution . . . 35
7.6 Budget global d'intervention . . . 36
7.7 Modèle d'optimisation conjointe déploiement–retrait ($\mathcal{P}_E$) . . . 36

8 Conclusion 38

Annexe : Mapping des données et alimentation du modèle avec les données du terrain 39

Bibliographie 41

1 INTRODUCTION

1.1 Contexte

Le secteur des télécommunications traverse une phase de transformation majeure en raison de l'explosion du trafic de données mobiles. Cette hausse est portée par la multiplication des terminaux connectés et l'usage massif de services gourmands en bande passante comme la vidéo en streaming. Pour le Groupe Orange, l'enjeu est de moderniser ses infrastructures pour accueillir ces nouveaux usages notamment en Afrique tout en tenant compte de contraintes fortes comme le budget. L'opérateur doit en effet arbitrer ses investissements selon des budgets annuels limités et des exigences réglementaires de l'ARCEP et ses équivalents en afrique concernant la couverture et la qualité de l'expérience utilisateur. Le déploiement d'une nouvelle technologie, comme la 5G, devient alors un levier stratégique pour capter de nouvelles parts de marché.

1.2 L'état de l'art

Les réseaux 3G/4G sont aujourd'hui largement présents, mais de larges écarts subsistent en zones rurales et périurbaines, notamment en termes de couverture, de qualité de service et de pénétration d'Internet haut débit : malgré le progrès continu, environ 20–30% de la population n'a toujours pas accès à des services 4G adéquats et la 5G reste très limitée (< 10 % de la population couverte à l'échelle continentale) [1]. De plus, les contraintes économiques (faible ARPU – revenu par abonné), l'insuffisance d'infrastructures physiques (tours, énergie fiable) et des environnements géographiques difficiles rendent l'optimisation des déploiements réseau cruciale pour réduire les coûts tout en maximisant la couverture et la capacité. Dans [2], les auteurs étudient la conception de stratégies d'investissement mobiles à coût minimal, en tenant compte de l'équilibrage de charge entre plusieurs technologies. Ils proposent une approche de résolution exacte basée sur la programmation linéaire en nombres entiers.

Les auteurs de [3] s'intéressent à un problème de planification des investissements sur plusieurs années et introduisent une formulation en programmation non linéaire qui intègre à la fois la dynamique du réseau et celle des abonnés. Cette formulation est ensuite linéarisée et renforcée par l'ajout d'inégalités valides afin de traiter des instances de petite à moyenne taille, tandis qu'une heuristique permet de générer des solutions réalisables de haute qualité pour des instances plus grandes.

Toutefois, ces approches restent principalement centrées sur la performance d'un opérateur pris isolément, alors que, dans la pratique, les décisions de déploiement s'inscrivent dans un environnement concurrentiel où les investissements d'un acteur influencent les comportements de migration des abonnés ainsi que les réactions stratégiques des autres opérateurs. Dans cette perspective, la thèse de Paolo Zappalà [10] montre que la théorie des jeux constitue un cadre particulièrement pertinent pour analyser les stratégies d'investissement dans les réseaux mobiles sous concurrence. Elle met en évidence l'intérêt des jeux non coopératifs pour modéliser l'interdépendance des décisions des opérateurs, ainsi que celui des jeux en forme extensive et des *discrete timing games* pour représenter la dimension séquentielle et dynamique des choix de déploiement. Cette littérature éclaire ainsi le positionnement de notre travail, qui ne cherche pas à résoudre l'ensemble du jeu concurrentiel, mais à déterminer la meilleure réponse d'un opérateur face à des stratégies concurrentes supposées données.

1.3 Cadre et organisation

Réalisé dans le cadre de la convention entre l'École Polytechnique et Orange, ce PSC porte sur l'élaboration d'un modèle définissant la "meilleure réponse" d'un opérateur face aux stratégies concurrentielles. Sous la direction de nos tuteurs, Matthieu Chardy, Amal Benhamiche, Youssouf Hadhbi et Aurélien Bechler, nous avons pu faire progresser significativement nos travaux.

1.4 Structure du manuscrit

Ce manuscrit structure la réflexion autour de l'optimisation du déploiement des réseaux mobiles de nouvelle génération (NG), en particulier dans le contexte africain, en articulant trois axes majeurs. Il débute par la formalisation mathématique d'un marché concurrentiel où l'opérateur cherche à maximiser sa part de marché sous contraintes budgétaires, réglementaires et de capacité, évoluant d'un modèle non linéaire complexe (MINLP) vers des versions linéaires (MILP) plus robustes et scalables. Le cœur du travail explore ensuite une hiérarchie de méthodes de résolution, allant d'approches exactes à des heuristiques innovantes (FixRelax, Algorithmes Génétiques et Mémétiques) et à l'usage de l'intelligence artificielle (Graph Neural Networks et Apprentissage par Renforcement) pour traiter des instances de taille industrielle en un temps de calcul raisonnable. Enfin, la réflexion s'élargit par une extension énergétique et environnementale, intégrant le retrait des technologies obsolètes (3G/4G) comme levier de gestion de la consommation et de l'empreinte carbone, offrant ainsi une vision holistique et durable de la stratégie réseau.

2
FORMALISATION DU PROBLÈME TÉLÉCOM

Nous considérons un marché compétitif, composé d'un ensemble fini d'opérateurs de réseaux mobiles noté I. Chaque opérateur $i \in I$ opère un ensemble de sites $S_i \quad i \in I$, sur lesquels sont déployées des technologies dites traditionnelles regroupées dans l'ensemble $G = \{2\text{G}, 3\text{G}, 4\text{G}\}$.

L'objectif de chaque opérateur est de moderniser progressivement son réseau en déployant une technologie mobile de nouvelle génération, notée NG(par exemple 5G ou 6G), au cours de l'horizon de planification.

La compétition entre opérateurs se joue sur une base de clients potentiels répartis dans un ensemble de zones géographiques A où chaque zone est indexée par $a \in A$. La population potentielle dans la zone a est notée u_a. Chaque opérateur $i \in I$ propose un ensemble d'offres commerciales associées aux technologies traditionnelles : $O_i = \{o_i^{2G}, o_i^{3G}, o_i^{4G}\}$, auxquelles s'ajoute une offre correspondant à la technologie de nouvelle génération, NO_i. L'ensemble global des offres disponibles sur le marché est ainsi donné par $O = \bigcup_{i \in I} (O_i \cup \{NO_i\})$.

Pour tout opérateur $i \in I$, toute zone $a \in A$, toute offre $o \in O_i \cup \{NO_i\}$, et toute période $t \in \mathcal{T}$, où $\mathcal{T} = \{0, 1, \ldots, |\mathcal{T}|\}$ désigne l'horizon discret de planification (par exemple en semestres ou en années), nous notons $u^t_{a,i,o}$ le nombre de clients situés dans la zone a souscrivant à l'offre o proposée par l'opérateur i à la période t. L'état initial du système, représentant la répartition des abonnés avant toute décision de déploiement, est noté $u^0_{a,i,o}$.

Cette variable évolue au fil du temps en fonction des investissements réalisés par chaque opérateur, des performances réseau qui en découlent, ainsi que des migrations de clients entre offres et entre opérateurs.
Hypothèse 1. Le nombre de clients potentiels dans une aire géographique est supposé constant au cours du temps.

Hypothèse 2. Les clients ne s'intéressent qu'à l'amélioration de leur offre, c'est-à-dire qu'ils souhaitent uniquement migrer vers une offre leur permettant d'accéder à la technologie NG.

Le but est de concevoir un plan de déploiement NG pour un opérateur cible noté $\tau \in I$ (dans notre cas, Orange). Ce plan est modélisé par un ensemble de variables de décision binaires $z_s^t, \quad s \in S_\tau, \ \mathrm{t} \in \mathcal{T}$, où $z_s^t = 1$ signifie que la technologie NG est installée sur le site s à l'instant t, et $z_s^t = 0$ sinon.

Hypothèse 3. Les plans de déploiement des concurrents pour l'ensemble $\bar{I} = I \setminus \{\tau\}$ sont supposés connus et sont modélisés par les paramètres $Z_{s,i}^t \in \{0,1\}, \quad s \in S_i, \ i \in \bar{I}, \ t \in \mathcal{T}$, où $Z_{s,i}^t = 1$ indique que l'opérateur concurrent i a déployé la technologie NG sur le site s à la période t.

Nous notons $\mathcal{A}_s$ l'ensemble des zones couvertes par le site s et $\mathcal{S}_{a,i}$ l'ensemble des sites de l'opérateur i qui couvrent la zone $a \in \mathcal{A}$, si la technologie NG y est installée.

Ainsi, l'état de couverture des différentes zones par l'opérateur τ est représenté par des variables binaires $r_a^t \in \{0,1\}, \quad a \in \mathcal{A}, \ t \in \mathcal{T}$, où $r_a^t = 1$ si la zone a est couverte par la technologie NG à l'instant t pour l'opérateur τ.
Pour les concurrents, l'état de couverture est modélisé par les paramètres $R_{a,i}^t, \quad a \in \mathcal{A}, \ i \in \bar{I}, \ t \in \mathcal{T}$.

Hypothèse 4. Nous supposons que les u_a clients potentiels de toute zone $a \in \mathcal{A}$ ont connaissance du statut de couverture de leur zone par tous les opérateurs. Ce statut est représenté, à chaque intervalle de temps $\mathrm{t} \in \mathcal{T}$, par le vecteur :

$$C_a^t = \langle r_a^t, (R_{a,i}^t)_{i \in \bar{I}} \rangle,$$

où r_a^t et $R_{a,i}^t$ représentent l'état de couverture par l'opérateur cible et par le concurrent $i \in \bar{I}$, respectivement.
Les clients décident de leur opérateur et de leur offre pour l'intervalle suivant $t+1$ selon un mécanisme de migration et de churn, formalisé par la fonction :

$$f_{C,o,o'}^a : \{0,1\}^{|I|} \times O \times O \longrightarrow [0,1],$$

qui indique le pourcentage de clients $u_{a,i,o}^t$ souscrivant à une offre $o \in O_i \cup \{NO_i\}$ de l'opérateur $i \in I$ qui migrent vers l'offre $NO_{i'}$ de l'opérateur $i' \in I$ si la condition de couverture $C_a^t = C$ est vérifiée. Cette condition peut être encodée à l'aide d'une variable binaire $\delta_{C,a}^t$.

Nous considérons que les sites équipés de la technologie NG disposent d'une capacité d'écoulement de traffic notée $CAPA_{NG}$. Chaque utilisateur d'une offre NO_τ a une demande de trafic moyenne D_{NG} .
Le traffic de ces utilisateurs peut être acheminé vers n'importe quel site de l'opérateur τ, à condition que le site couvre la zone de l'utilisateur et que la technologie NG y soit installée.

Nous notons $u_{a,\tau,NO_\tau,s}^t$ le nombre d'utilisateurs de l'offre NO_τ situés dans la zone $a \in \mathcal{A}$ de l'opérateur τ, qui communiquent à travers le site $s \in \mathcal{S}_{a,\tau}$, à l'intervalle de temps $t \in \mathcal{T}$.

Notre objectif est de maximiser la part de marché de notre opérateur $\tau \in I$ pour son offre NO_τ à la fin de l'horizon temporel considéré. Le plan de déploiement de la technologie NG de τ est soumis à plusieurs contraintes opérationnelles et budgétaires :

- **Contraintes budgétaires :** le nombre maximal de sites pouvant être équipés de NG à chaque intervalle de temps $t \in \mathcal{T}$ est limité et noté Z^t ;
- **Contraintes réglementaires :** le classement annuel des opérateurs par les régulateurs (par exemple ARCEP en France) impose des directives stratégiques, telles que des pourcentages minimaux de la population devant être couverts par la technologie NG, notés QA^t pour chaque intervalle $t \in \mathcal{T}$;
- **Contraintes opérationnelles :** la planification doit respecter la capacité des sites $CAPA_{NG}$ et l'allocation des utilisateurs $u_{a,\tau,NO_\tau,s}^t$ à travers les sites disponibles ;

3
MODÈLES MATHÉMATIQUES

3.1 Modèle initial (MINLP^{initial})

D'après le rapport étudié [4], le problème est formulé comme un programme non linéaire en nombres entiers mixtes (MINLP^{initial}).

Le modèle mathématique résultant est composé d'une fonction objectif maximisant la part de marché de l'offre NO de l'opérateur d'intérêt, et d'un ensemble de contraintes traduisant les règles de couverture, de migration des utilisateurs, ainsi que des contraintes budgétaires et réglementaires. Cette formulation est non linéaire et ne peut pas être résolue directement par des solveurs MILP.

La formulation complète est donnée ci-dessous :

$$\max_{z,u,r} \sum_{a\in\mathcal{A}} u^{t_{|\mathcal{T}|}}_{a,\tau,NO_\tau} \tag{1.1}$$

$$st.$$

$$r^t_a \le \sum_{s\in\mathcal{S}_{a,\tau}} z^t_s \qquad \forall a\in\mathcal{A}, t\in\mathcal{T}, \tag{1.2}$$

$$z^t_s \le r^t_a \qquad \forall a\in\mathcal{A}, s\in\mathcal{S}_{a,\tau}, t\in\mathcal{T}, \tag{1.3}$$

$$\delta^t_{a,C} = (c_\tau r^t_a + (1-c_\tau)(1-r^t_a)) \prod_{i\in\overline{I}} (c_i R^t_{a,i} + (1-c_i)(1-R^t_{a,i})) \qquad \forall C=(c_i)_{i\in I}\in\{0;1\}^{|I|}, a\in\mathcal{A}, t\in\mathcal{T}, \tag{1.4}$$

$$u^t_{a,i,o} = \sum_{C\in\{0;1\}^{|I|}} \delta^t_{a,C} \sum_{i'\in I} \sum_{o'\in\mathcal{O}_{i'}\cup\{NO_{i'}\}} f^a_{C,o',o} \cdot u^{t-1}_{a,i',o'} \qquad \forall a\in\mathcal{A}, i\in I, o\in\mathcal{O}_i\cup\{NO_i\}, t\in\mathcal{T}\setminus\{0\}, \tag{1.5}$$

$$u^t_{a,\tau,NO_\tau} = \sum_{s\in\mathcal{S}_{a,\tau}} u^t_{a,\tau,NO_\tau,s} \qquad \forall a\in\mathcal{A}, t\in\mathcal{T}, \tag{1.6}$$

$$\sum_{a\in\mathcal{A}_s} D_{NG}\cdot u^t_{a,\tau,NO_\tau,s} \le CAPA_{NG}\cdot z^t_s \qquad \forall s\in\mathcal{S}, t\in\mathcal{T}, \tag{1.7}$$

$$\sum_{s\in\mathcal{S}_\tau} (z^t_s - z^{t-1}_s) \le \overline{Z}^t \qquad \forall t\in\mathcal{T}\setminus\{0\}, \tag{1.8}$$

$$\sum_{a\in\mathcal{A}} u_a r^t_a \ge \underline{QA}^t \cdot \sum_{a\in\mathcal{A}} u_a \qquad \forall t\in\mathcal{T}, \tag{1.9}$$

$$r^t_a \in \{0;1\} \qquad \forall a\in\mathcal{A}, t\in\mathcal{T}, \tag{1.10}$$

$$z^t_s \in \{0;1\} \qquad \forall s\in\mathcal{S}, t\in\mathcal{T}, \tag{1.11}$$

$$\delta^t_{a,C} \in \{0;1\} \qquad \forall C\in\{0;1\}^{|I|}, a\in\mathcal{A}, t\in\mathcal{T}, \tag{1.12}$$

$$u^t_{a,i,o} \in \mathbb{N} \qquad \forall a\in\mathcal{A}, i\in I, o\in\mathcal{O}_i\cup\{NO_i\}, t\in\mathcal{T}, \tag{1.13}$$

$$u^t_{a,\tau,NO_\tau,s} \in \mathbb{N} \qquad \forall a\in\mathcal{A}, s\in\mathcal{S}_{a,\tau}, t\in\mathcal{T}. \tag{1.14}$$

Fonction objectif. L'équation (1.1) définit la fonction objectif du modèle. L'objectif est de maximiser le nombre total de clients souscrivant à l'offre NO_τ de l'opérateur d'intérêt τ, sur l'ensemble des zones $a\in\mathcal{A}$, à l'horizon temporel final $t_{|\mathcal{T}|}$. Autrement dit, on cherche à maximiser la part de marché de l'opérateur τ à la fin de la période de planification correspondant à la stratégie de déploiement de la technologie de nouvelle génération.

Contraintes de couverture couplantes. Les contraintes (1.2) et (1.3) expriment des contraintes logiques reliant la variable de couverture r_a^t aux décisions de déploiement z_s^t. La contrainte (2) garantit que si $r_a^t = 1$, alors la zone a est couverte par la technologie de nouvelle génération à l'instant t, ce qui implique l'existence d'au moins un site $s \in S_{a,\tau}$ tel que $z_s^t = 1$.

Inversement, la contrainte (1.3) impose que si $r_a^t = 0$, alors aucun site couvrant la zone a n'est équipé de la technologie de nouvelle génération, c'est-à-dire que $z_s^t = 0$ pour tout $s \in S_{a,\tau}$. Ces deux contraintes assurent ainsi une cohérence logique entre la couverture des zones et le déploiement effectif des sites.

Identification du vecteur de couverture. L'équation (1.4) permet d'identifier le vecteur de couverture $C = (c_i)_{i \in I}$ associé à la zone a à l'instant t. La variable binaire $\delta_{a,C}^t$ est égale à 1 si et seulement si le vecteur C correspond exactement à l'état de couverture réel des opérateurs dans la zone a à l'instant t, c'est-à-dire si $C = \langle r_a^t, (R_{a,i}^t)_{i \in \bar{I}} \rangle$. Dans le cas contraire, $\delta_{a,C}^t$ est égale à 0. Cette variable joue ainsi le rôle d'un indicateur de correspondance entre l'état de couverture observé et un scénario donné.

Dynamique de migration des clients. L'équation (1.5) modélise la dynamique temporelle de migration des clients entre les différentes offres et les différents opérateurs. Le nombre de clients $u_{a,i,o}^t$ correspond au nombre de clients présents dans la zone a, abonnés à l'offre o de l'opérateur i à l'instant t.

Cette population résulte des migrations des clients présents à l'instant $t-1$, chaque client initialement abonné à l'offre o' de l'opérateur i' pouvant migrer vers l'offre o de l'opérateur i selon une probabilité (ou un pourcentage) donnée par la fonction de migration $f_{C,o',o}^a$, laquelle dépend du vecteur de couverture C. La somme sur l'ensemble des scénarios de couverture permet de prendre en compte l'impact du déploiement des technologies sur les choix des clients.

Répartition des clients sur les sites. L'équation (1.6) exprime que le nombre total de clients utilisant l'offre NO_τ de l'opérateur τ dans la zone a est égal à la somme des clients connectés à chacun des sites $s \in S_{a,\tau}$ couvrant cette zone. Ainsi, les clients d'une même zone peuvent être répartis sur plusieurs sites équipés de la technologie de nouvelle génération, ce qui permet ensuite de modéliser correctement les contraintes de capacité du réseau.

Contraintes de capacité du réseau. La contrainte (1.7) garantit le respect des capacités des sites équipés de la technologie de nouvelle génération. Lorsqu'un site s est équipé de la NG à l'instant t, c'est-à-dire lorsque $z_s^t = 1$, la somme des demandes de trafic générées par les clients NO_τ connectés à ce site ne doit pas dépasser sa capacité maximale CAPA_{NG}. Chaque client ayant une demande moyenne D_{NG}, le membre de gauche de l'inégalité représente la charge totale de trafic supportée par le site s, laquelle doit rester inférieure à la capacité disponible.

Contraintes budgétaires de déploiement. La contrainte (1.8) limite le nombre de nouveaux sites pouvant être équipés de la technologie de nouvelle génération à chaque période t. Elle impose que la différence entre le nombre de sites équipés à l'instant t et ceux déjà équipés à l'instant $t-1$ soit inférieure ou égale à $\overline{Z}^t$, qui représente le nombre maximal de sites pouvant être déployés sur une période. Cette contrainte modélise ainsi la contrainte budgétaire pesant sur la stratégie de déploiement de l'opérateur.

Contraintes réglementaires de couverture. La contrainte (1.9) traduit une exigence réglementaire imposée par les autorités. Le membre de gauche correspond au nombre total de clients situés dans des zones couvertes par la technologie de nouvelle génération à l'instant t. Rapportée à la population totale $\sum_{a \in \mathcal{A}} u_a$, cette quantité représente le pourcentage de la population bénéficiant de la NG. Cette proportion doit être supérieure ou égale à QA^t, qui désigne le seuil minimal de couverture imposé par la réglementation à l'instant t.

Contraintes d'intégrité des variables de couverture. Les contraintes (1.10) et (1.11) définissent respectivement les variables r_a^t et z_s^t comme des variables binaires. La variable r_a^t indique si la zone a est couverte par la NG à l'instant t, tandis que z_s^t indique si le site s est équipé de la NG à l'instant t.

Contraintes d'intégrité des variables auxiliaires. La contrainte (1.12) impose que la variable $\delta^t_{a,C}$ soit binaire. Les contraintes (1.13) et (1.14)définissent les variables $u^t_{a,i,o}$ et $u^t_{a,i,o,s}$ comme des variables entières naturelles, représentant respectivement le nombre de clients par zone, opérateur et offre, et leur répartition sur les sites du réseau.

3.2 Améliorations et renforcements ($\text{MILP}^{initial}$)

À l'issue de l'étude du modèle mathématique initial proposé dans le rapport, nous avons envisagé plusieurs modifications afin d'améliorer sa cohérence mathématique et son adéquation avec les données du problème.

3.2.1 • Linéarisation de la contrainte de migration.

La contrainte (1.5), qui modélise la dynamique de migration des clients entre offres et opérateurs, est non linéaire : elle fait intervenir le produit d'une variable binaire $\delta^t_{a,C}$ et d'une expression linéaire en $u^{t-1}_{a,i',o'}$. Pour la rendre compatible avec un solveur MILP, nous la linéarisons par la technique classique du *Big-M*.

On introduit d'abord la variable auxiliaire

$$S^t_{a,i,C,o} = \sum_{i' \in I} \sum_{o' \in \mathcal{O}_{i'} \cup \{NO_{i'}\}} f^a_{C,o',o}\, u^{t-1}_{a,i',o'},$$

bornée par $0 \le S^t_{a,i,C,o} \le M_{a,C,o}$ avec

$$M_{a,C,o} = \sum_{i' \in I} \sum_{o' \in \mathcal{O}_{i'} \cup \{NO_{i'}\}} u^0_{a,i',o'}.$$

On pose ensuite

$$y^t_{a,i,C,o} = \delta^t_{a,C}\, S^t_{a,i,C,o},$$

de sorte que la contrainte de migration s'écrit :

$$u^t_{a,i,o} = \sum_{C \in \{0,1\}^{|I|}} y^t_{a,i,C,o}, \qquad \forall\, t \in \mathcal{T} \setminus \{0\}.$$

Le produit binaire-continu $y = \delta \cdot S$ est alors linéarisé par les contraintes suivantes :

$$y^t_{a,i,C,o} \le M_{a,C,o}\, \delta^t_{a,C}, \qquad y^t_{a,i,C,o} \le S^t_{a,i,C,o}, \qquad y^t_{a,i,C,o} \ge S^t_{a,i,C,o} - M_{a,C,o}(1 - \delta^t_{a,C}), \qquad y^t_{a,i,C,o} \ge 0,$$

pour tout $a \in \mathcal{A}$, $i \in I$, $o \in \mathcal{O}_i \cup \{NO_i\}$, $C \in \{0,1\}^{|I|}$ et $t \in \mathcal{T}$.

Le modèle devient ainsi entièrement linéaire en les variables de décision, au prix de l'introduction d'une variable auxiliaire et de quatre contraintes supplémentaires par tuple (t, a, i, C, o).

3.2.2 • Dépendance temporelle des paramètres de trafic.

Par ailleurs, l'analyse des données du problème a révélé que la demande moyenne de trafic par client D_{NG} ainsi que la capacité des sites CAPA_{NG} évoluent au cours du temps. Nous avons donc modifié le modèle en considérant ces deux paramètres comme dépendant de l'instant t, ce qui permet de mieux représenter l'évolution technologique et l'augmentation progressive des usages.

3.2.3 • Monotonicité du déploiement des sites NG.

Le modèle initial n'imposait aucune contrainte garantissant qu'un site équipé de la technologie de nouvelle génération le reste dans le temps. Il était donc théoriquement possible d'avoir $z^t_s = 1$ et $z^{t+1}_s = 0$, ce qui n'est pas réaliste d'un point de vue opérationnel. En pratique, une fois un site équipé de la NG, celui-ci reste équipé pour les périodes suivantes. Afin de corriger ce point, nous avons ajouté la contrainte de monotonie suivante :

$$z^t_s \ge z^{t-1}_s, \quad \forall s \in \mathcal{S},\ \forall t \in \mathcal{T} \setminus \{0\}.$$

3.2.4 • Relaxation de l'intégrité des variables de population.

Enfin, nous avons remarqué que, dans la contrainte (1.5), les variables $u^t_{a,i,o}$ ne sont pas nécessairement entières après la migration des clients. On considère donc $u^t_{a,i,o}$ et $u^t_{a,i,o,s}$ comme des variables réelles positives.

3.2.5 • MILP^{initial}

$$\max_{z,u,r,\delta,S,y} \quad \sum_{a\in\mathcal{A}} u^{t_{|\mathcal{T}|}}_{a,\tau,NO_\tau} \tag{2.1}$$

s.t.

$$r^t_a \le \sum_{s\in\mathcal{S}_{a,\tau}} z^t_s \qquad \forall a\in\mathcal{A},\ t\in\mathcal{T} \tag{2.2}$$

$$z^t_s \le r^t_a \qquad \forall a\in\mathcal{A},\ s\in\mathcal{S}_{a,\tau},\ t\in\mathcal{T} \tag{2.3}$$

$$\delta^t_{a,C} = \big(c_\tau r^t_a + (1-c_\tau)(1-r^t_a)\big) \prod_{i\in\overline{I}}\big(c_i R^t_{a,i} + (1-c_i)(1-R^t_{a,i})\big) \qquad \forall C\in\{0,1\}^{|I|},\ a\in\mathcal{A},\ t\in\mathcal{T} \tag{2.4}$$

$$S^t_{a,i,C,o} = \sum_{i'\in I}\sum_{o'\in\mathcal{O}_{i'}\cup\{NO_{i'}\}} f^a_{C,o',o}\cdot u^{t-1}_{a,i',o'} \qquad \forall t\in\mathcal{T}\setminus\{0\},\ a\in\mathcal{A},\ i\in I,\ C\in\{0,1\}^{|I|},\ o\in\mathcal{O}_i\cup\{NO_i\} \tag{2.5}$$

$$y^t_{a,i,C,o} = \delta^t_{a,C}\cdot S^t_{a,i,C,o} \qquad \forall t\in\mathcal{T}\setminus\{0\},\ a\in\mathcal{A},\ i\in I,\ C\in\{0,1\}^{|I|},\ o\in\mathcal{O}_i\cup\{NO_i\} \tag{2.6}$$

$$u^t_{a,i,o} = \sum_{C\in\{0,1\}^{|I|}} y^t_{a,i,C,o} \qquad \forall t\in\mathcal{T}\setminus\{0\},\ a\in\mathcal{A},\ i\in I,\ o\in\mathcal{O}_i\cup\{NO_i\} \tag{2.7}$$

$$0 \le S^t_{a,i,C,o} \le M_{a,C,o} \qquad \forall t\in\mathcal{T}\setminus\{0\},\ a\in\mathcal{A},\ i\in I,\ C\in\{0,1\}^{|I|},\ o\in\mathcal{O}_i\cup\{NO_i\} \tag{2.8}$$

$$y^t_{a,i,C,o} \le M_{a,C,o}\cdot\delta^t_{a,C} \qquad \forall t\in\mathcal{T}\setminus\{0\},\ a\in\mathcal{A},\ i\in I,\ C\in\{0,1\}^{|I|},\ o\in\mathcal{O}_i\cup\{NO_i\} \tag{2.9}$$

$$y^t_{a,i,C,o} \le S^t_{a,i,C,o} \qquad \forall t\in\mathcal{T}\setminus\{0\},\ a\in\mathcal{A},\ i\in I,\ C\in\{0,1\}^{|I|},\ o\in\mathcal{O}_i\cup\{NO_i\} \tag{2.10}$$

$$y^t_{a,i,C,o} \ge S^t_{a,i,C,o} - M_{a,C,o}(1-\delta^t_{a,C}) \qquad \forall t\in\mathcal{T}\setminus\{0\},\ a\in\mathcal{A},\ i\in I,\ C\in\{0,1\}^{|I|},\ o\in\mathcal{O}_i\cup\{NO_i\} \tag{2.11}$$

$$y^t_{a,i,C,o} \ge 0 \qquad \forall t\in\mathcal{T}\setminus\{0\},\ a\in\mathcal{A},\ i\in I,\ C\in\{0,1\}^{|I|},\ o\in\mathcal{O}_i\cup\{NO_i\} \tag{2.12}$$

$$u^t_{a,\tau,NO_\tau} = \sum_{s\in\mathcal{S}_{a,\tau}} u^t_{a,\tau,NO_\tau,s} \qquad \forall a\in\mathcal{A},\ t\in\mathcal{T} \tag{2.13}$$

$$\sum_{a\in\mathcal{A}_s} D^t_{NG}\cdot u^t_{a,\tau,NO_\tau,s} \le CAPA^t_{NG}\cdot z^t_s \qquad \forall s\in\mathcal{S},\ t\in\mathcal{T} \tag{2.14}$$

$$z^t_s \ge z^{t-1}_s \qquad \forall s\in\mathcal{S},\ t\in\mathcal{T}\setminus\{0\} \tag{2.15}$$

$$\sum_{s\in\mathcal{S}_\tau}(z^t_s - z^{t-1}_s) \le \overline{Z}^t \qquad \forall t\in\mathcal{T}\setminus\{0\} \tag{2.16}$$

$$\sum_{a\in\mathcal{A}} u_a r^t_a \ge \underline{QA}^t\cdot\sum_{a\in\mathcal{A}} u_a \qquad \forall t\in\mathcal{T} \tag{2.17}$$

$$r^t_a\in\{0,1\} \qquad \forall a\in\mathcal{A},\ t\in\mathcal{T} \tag{2.18}$$

$$z^t_s\in\{0,1\} \qquad \forall s\in\mathcal{S},\ t\in\mathcal{T} \tag{2.19}$$

$$\delta^t_{a,C}\in\{0,1\} \qquad \forall C\in\{0,1\}^{|I|},\ a\in\mathcal{A},\ t\in\mathcal{T} \tag{2.20}$$

$$u^t_{a,i,o}\in\mathbb{R}_+ \qquad \forall a\in\mathcal{A},\ i\in I,\ o\in\mathcal{O}_i\cup\{NO_i\},\ t\in\mathcal{T} \tag{2.21}$$

$$S^t_{a,i,C,o}\in\mathbb{R}_+ \qquad \forall t\in\mathcal{T}\setminus\{0\},\ a\in\mathcal{A},\ i\in I,\ C\in\{0,1\}^{|I|},\ o\in\mathcal{O}_i\cup\{NO_i\} \tag{2.22}$$

$$y^t_{a,i,C,o}\in\mathbb{R}_+ \qquad \forall t\in\mathcal{T}\setminus\{0\},\ a\in\mathcal{A},\ i\in I,\ C\in\{0,1\}^{|I|},\ o\in\mathcal{O}_i\cup\{NO_i\} \tag{2.23}$$

$$u^t_{a,\tau,NO_\tau,s}\in\mathbb{R}_+ \qquad \forall a\in\mathcal{A},\ s\in\mathcal{S}_{a,\tau},\ t\in\mathcal{T} \tag{2.24}$$

3.2.6 • Amélioration du choix du paramètre Big-M

Dans la linéarisation

$$y^t_{a,i,C,o} = \delta^t_{a,C}\, S^t_{a,i,C,o},$$

le paramètre $M_{a,C,o}$ doit majorer $S^t_{a,i,C,o}$ pour chaque configuration C. Une borne uniforme indépendante de C est valide mais trop conservative ; on construit donc une borne dépendant explicitement de C.

On rappelle que

$$S^t_{a,i,C,o} = \sum_{i' \in I} \sum_{o' \in \mathcal{O}_{i'} \cup \{NO_{i'}\}} f^a_{C,o',o}\, u^{t-1}_{a,i',o'},$$

avec $0 \leq f^a_{C,o',o} \leq 1$. Comme la population totale par zone est conservée,

$$\sum_{i' \in I} \sum_{o' \in \mathcal{O}_{i'} \cup \{NO_{i'}\}} u^{t-1}_{a,i',o'} = \sum_{i' \in I} \sum_{o' \in \mathcal{O}_{i'} \cup \{NO_{i'}\}} u^{0}_{a,i',o'},$$

d'où la majoration valide pour tout t :

$$M_{a,C,o} = (\max_{o' \in \mathcal{O}} f^a_{C,o',o}) \cdot \sum_{i' \in I} \sum_{o' \in \mathcal{O}_{i'} \cup \{NO_{i'}\}} u^0_{a,i',o'}. \tag{1}$$

Strictement plus précise qu'une borne uniforme, elle resserre la relaxation linéaire et améliore les performances du solveur (temps de résolution et qualité des bornes), notamment lors du Branch and Bound.

3.3 Reformulation (MILP^{reformulé})

3.3.1 • Réduction de la contrainte (1.4) de décodage de l'indicatrice

La contrainte (1.4) du modèle initial avait pour rôle de *décoder* la configuration $C = (c_i)_{i \in I}$ à partir du couple (r^t_a, R^t_a), en imposant $\delta^t_{a,C} = 1$ si et seulement si $C = \langle r^t_a, R^t_a \rangle$. Dans le modèle complet, cette logique était encodée par le produit booléen

$$\delta^t_{a,C} = \left(c_\tau r^t_a + (1 - c_\tau)(1 - r^t_a)\right) \prod_{i \in \overline{I}} \left(c_i R^t_{a,i} + (1 - c_i)(1 - R^t_{a,i})\right),$$

qui réalise une *filtrage dynamique* sur $\{0,1\}^{|I|}$: seules les configurations compatibles avec (r^t_a, R^t_a) survivent, toutes les autres sont annulées.

Or R^t_a étant un *paramètre fixé*, il est inutile de reconstruire dynamiquement cet ensemble : il suffit de filtrer *a priori* les C dont les composantes $(c_i)_{i \neq \tau}$ coïncident avec $R^t_{a,i}$. La seule composante réellement variable est c_τ, contrainte de valoir r^t_a. Les contraintes (1.4) et (1.5) se réduisent alors à une unique équation de transition intégrant les deux seules configurations possibles $C = \langle 1, R^t_a \rangle$ et $C = \langle 0, R^t_a \rangle$:

$$u^t_{a,i,o} = r^t_a \sum_{i',o'} f^a_{(1,R^t_a),o',o}\, u^{t-1}_{a,i',o'} + (1 - r^t_a) \sum_{i',o'} f^a_{(0,R^t_a),o',o}\, u^{t-1}_{a,i',o'}. \tag{2}$$

Amélioration en termes de taille du modèle. La simplification a un impact majeur sur la taille du MILP. Dans le modèle initial, la contrainte de décodage (1.4) imposait une variable binaire $\delta^t_{a,C}$ pour *chaque* configuration $C \in \{0,1\}^{|I|}$, chaque actif a et chaque période t, soit un nombre total d'ordre $|\mathcal{A}| \cdot |\mathcal{T}| \cdot 2^{|I|}$, avec autant de contraintes Big-M associées pour relier $\delta^t_{a,C}$, $S_{t,a,C,o}$ et $y_{t,a,C,o}$. Cette explosion combinatoire dominait la taille du modèle et en constituait un principal facteur limitant.

En remplaçant (1.4) par la transition explicite à deux cas $c_\tau \in \{0,1\}$, on élimine entièrement les variables $\delta^t_{a,C}$ et leurs contraintes de linéarisation. Le nombre de variables et d'équations est ainsi réduit d'un facteur exponentiel, et le modèle final ne conserve que les variables essentielles $(r^t_a, z^t_s, u^t_{a,i,o})$. La scalabilité du MILP s'en trouve drastiquement améliorée, permettant de traiter des instances de taille industrielle.

Aprés linéarisation le MINLP^{reformulé} résultant aura un nombre réduit significativement de contrainte à linéariser.

3.3.2 • Linéarisation du nouveau modéle

Dans le but de préparer la simplification du modèle et d'isoler explicitement le terme bilinéaire présent dans la dynamique de transition, nous reformulons la contrainte (2) sous la forme :

$$u^t_{a,i,o} = S^t_{a,i,o} + \sum_{i' \in I} \sum_{o' \in \mathcal{O}_{i'} \cup \{NO_{i'}\}} f^a_{(0,R^t_a),o',o} \cdot u^{t-1}_{a,i',o'}, \tag{3}$$

avec

$$S^t_{a,i,o} = r^t_a \left(\sum_{i' \in I} \sum_{o' \in \mathcal{O}_{i'} \cup \{NO_{i'}\}} \left(f^a_{(1,R^t_a),o',o} - f^a_{(0,R^t_a),o',o} \right) \cdot u^{t-1}_{a,i',o'} \right) \tag{4}$$

Nous introduisons ainsi la variable auxiliaire $S^t_{a,i,o}$. Celle-ci représente la partie du flux de transition qui dépend de l'activation de l'actif, c'est-à-dire du choix binaire r^t_a. Plus précisément, $S^t_{a,i,o}$ capture la contribution associée au cas $r^t_a = 1$, tandis que le reste de la dynamique est regroupé dans un terme indépendant de r^t_a.

Choix des bornes de linéarisation Dans le deuxième modèle, la non-linéarité provient du terme bilinéaire $r^t_a \, \sigma^t_{a,i,o}$, où

$$\sigma^t_{a,i,o} = \sum_{i' \in I} \sum_{o' \in \mathcal{O}_{i'} \cup \{NO_{i'}\}} \left(f^a_{(1,R^t_a),o',o} - f^a_{(0,R^t_a),o',o} \right) u^{t-1}_{a,i',o'} \tag{5}$$

Afin de linéariser ce produit, on introduit la variable auxiliaire $S^t_{a,i,o} = r^t_a \sigma^t_{a,i,o}$ et on encadre $\sigma^t_{a,i,o}$ par deux bornes asymétriques $L^t_{a,i,o}$ et $U^t_{a,i,o}$ telles que

$$L^t_{a,i,o} \leq \sigma^t_{a,i,o} \leq U^t_{a,i,o} \tag{6}$$

Ces bornes sont construites à partir des variations extrêmes des coefficients de migration entre les deux états de couverture $r^t_a = 1$ et $r^t_a = 0$, ainsi que d'une majoration de la population totale de la zone a. On pose donc :

$$U^t_{a,i,o} = \max\left(0, \max_{i',\, o'} \left(f^a_{(1,R^t_a),o',o} - f^a_{(0,R^t_a),o',o} \right)\right) \sum_{i' \in I} \sum_{o' \in \mathcal{O}_{i'} \cup \{NO_{i'}\}} u^0_{a,i',o'} \tag{7}$$

et

$$L^t_{a,i,o} = \min\left(0, \min_{i',\, o'} \left(f^a_{(1,R^t_a),o',o} - f^a_{(0,R^t_a),o',o} \right)\right) \sum_{i' \in I} \sum_{o' \in \mathcal{O}_{i'} \cup \{NO_{i'}\}} u^0_{a,i',o'}. \tag{8}$$

Ainsi, au lieu d'utiliser une constante unique M et $-M$, on exploite un encadrement plus fin de $\sigma^t_{a,i,o}$. Cette formulation renforce la relaxation linéaire, réduit le conservatisme de la linéarisation et améliore en pratique le comportement numérique du solveur.

Linéarisation du produit $r^t_a \sigma^t_{a,i,o}$ La relation bilinéaire

$$S^t_{a,i,o} = r^t_a \sigma^t_{a,i,o}$$

est remplacée par les quatre contraintes linéaires suivantes :

$$S^t_{a,i,o} \leq U^t_{a,i,o} \, r^t_a \qquad \forall a \in \mathcal{A},\ i \in I,\ o \in \mathcal{O}_i \cup \{NO_i\},\ t \in \mathcal{T} \setminus \{0\}, \tag{9}$$

$$S^t_{a,i,o} \geq L^t_{a,i,o} \, r^t_a \qquad \forall a \in \mathcal{A},\ i \in I,\ o \in \mathcal{O}_i \cup \{NO_i\},\ t \in \mathcal{T} \setminus \{0\}, \tag{10}$$

$$S^t_{a,i,o} \leq \sigma^t_{a,i,o} - L^t_{a,i,o}(1 - r^t_a) \qquad \forall a \in \mathcal{A},\ i \in I,\ o \in \mathcal{O}_i \cup \{NO_i\},\ t \in \mathcal{T} \setminus \{0\}, \tag{11}$$

$$S^t_{a,i,o} \geq \sigma^t_{a,i,o} - U^t_{a,i,o}(1 - r^t_a) \qquad \forall a \in \mathcal{A},\ i \in I,\ o \in \mathcal{O}_i \cup \{NO_i\},\ t \in \mathcal{T} \setminus \{0\}. \tag{12}$$

Ces contraintes garantissent que :

$$S^t_{a,i,o} = \begin{cases} 0, & \text{si } r^t_a = 0, \\ \sigma^t_{a,i,o}, & \text{si } r^t_a = 1. \end{cases}$$

On obtient alors la reformulation MILP^{reformulé} suivante :

$$\max_{z,u,r,S} \quad \sum_{a\in\mathcal{A}} u^{t_{|\mathcal{T}|}}_{a,\tau,NO_\tau} \tag{3.1}$$

s.t.

$$r^t_a \le \sum_{s\in\mathcal{S}_{a,\tau}} z^t_s \qquad \forall a\in\mathcal{A},\ t\in\mathcal{T}, \tag{3.2}$$

$$z^t_s \le r^t_a \qquad \forall a\in\mathcal{A},\ s\in\mathcal{S}_{a,\tau},\ t\in\mathcal{T}, \tag{3.3}$$

$$u^t_{a,i,o} = S^t_{a,i,o} + \sum_{i'\in I}\sum_{o'\in\mathcal{O}_{i'}\cup\{NO_{i'}\}} f^a_{(0,R^t_a),o',o}\, u^{t-1}_{a,i',o'} \qquad \forall a\in\mathcal{A},\ i\in I,\ o\in\mathcal{O}_i\cup\{NO_i\},\ t\in\mathcal{T}\setminus\{0\}, \tag{3.4}$$

$$\sigma^t_{a,i,o} = \sum_{i'\in I}\sum_{o'\in\mathcal{O}_{i'}\cup\{NO_{i'}\}} \left(f^a_{(1,R^t_a),o',o} - f^a_{(0,R^t_a),o',o}\right) u^{t-1}_{a,i',o'} \qquad \forall a\in\mathcal{A},\ i\in I,\ o\in\mathcal{O}_i\cup\{NO_i\},\ t\in\mathcal{T}\setminus\{0\}, \tag{3.5}$$

$$S^t_{a,i,o} \le U^t_{a,i,o}\, r^t_a \qquad \forall a\in\mathcal{A},\ i\in I,\ o\in\mathcal{O}_i\cup\{NO_i\},\ t\in\mathcal{T}\setminus\{0\}, \tag{3.6}$$

$$S^t_{a,i,o} \ge L^t_{a,i,o}\, r^t_a \qquad \forall a\in\mathcal{A},\ i\in I,\ o\in\mathcal{O}_i\cup\{NO_i\},\ t\in\mathcal{T}\setminus\{0\}, \tag{3.7}$$

$$S^t_{a,i,o} \le \sigma^t_{a,i,o} - L^t_{a,i,o}(1-r^t_a) \qquad \forall a\in\mathcal{A},\ i\in I,\ o\in\mathcal{O}_i\cup\{NO_i\},\ t\in\mathcal{T}\setminus\{0\}, \tag{3.8}$$

$$S^t_{a,i,o} \ge \sigma^t_{a,i,o} - U^t_{a,i,o}(1-r^t_a) \qquad \forall a\in\mathcal{A},\ i\in I,\ o\in\mathcal{O}_i\cup\{NO_i\},\ t\in\mathcal{T}\setminus\{0\}, \tag{3.9}$$

$$u^t_{a,\tau,NO_\tau} = \sum_{s\in\mathcal{S}_{a,\tau}} u^t_{a,\tau,NO_\tau,s} \qquad \forall a\in\mathcal{A},\ t\in\mathcal{T}\setminus\{0\}, \tag{3.10}$$

$$\sum_{a\in\mathcal{A}_s} D^t_{NG}\, u^t_{a,\tau,NO_\tau,s} \le CAPA^t_{NG}\, z^t_s \qquad \forall s\in\mathcal{S}_\tau,\ t\in\mathcal{T}, \tag{3.11}$$

$$\sum_{s\in\mathcal{S}_\tau} (z^t_s - z^{t-1}_s) \le \overline{Z}^t \qquad \forall t\in\mathcal{T}\setminus\{0\}, \tag{3.12}$$

$$z^t_s \ge z^{t-1}_s \qquad \forall s\in\mathcal{S},\ t\in\mathcal{T}\setminus\{0\}, \tag{3.13}$$

$$\sum_{a\in\mathcal{A}} u_a r^t_a \ge \underline{QA^t}\cdot\sum_{a\in\mathcal{A}} u_a \qquad \forall t\in\mathcal{T}, \tag{3.14}$$

$$r^t_a \in \{0,1\} \qquad \forall a\in\mathcal{A},\ t\in\mathcal{T}, \tag{3.15}$$

$$z^t_s \in \{0,1\} \qquad \forall s\in\mathcal{S},\ t\in\mathcal{T}, \tag{3.16}$$

$$S^t_{a,i,o} \in \mathbb{R} \qquad \forall a\in\mathcal{A},\ i\in I,\ o\in\mathcal{O}_i\cup\{NO_i\},\ t\in\mathcal{T}, \tag{3.17}$$

$$\sigma^t_{a,i,o} \in \mathbb{R} \qquad \forall a\in\mathcal{A},\ i\in I,\ o\in\mathcal{O}_i\cup\{NO_i\},\ t\in\mathcal{T}, \tag{3.18}$$

$$u^t_{a,i,o} \in \mathbb{R}_+ \qquad \forall a\in\mathcal{A},\ i\in I,\ o\in\mathcal{O}_i\cup\{NO_i\},\ t\in\mathcal{T}, \tag{3.19}$$

$$u^t_{a,\tau,NO_\tau,s} \in \mathbb{R}_+ \qquad \forall a\in\mathcal{A},\ s\in\mathcal{S}_\tau,\ t\in\mathcal{T}. \tag{3.20}$$

Comparaison des bornes de linéarisation Dans le premier modèle, la linéarisation reposait sur une constante Big-M associée à un flux total de la forme

$$S^t_{i,a,C,o} = \sum_{i'\in I}\sum_{o'\in\mathcal{O}} f^a_{C,o',o}\, u^{t-1}_{a,i',o'}, \tag{2.5}$$

et conduisait à une borne du type

$$M^{(1)}_{a,C,o} = (\max_{o'} f^a_{C,o',o})\cdot\sum_{i'\in I}\sum_{o'\in\mathcal{O}} u^0_{a,i',o'}. \tag{13}$$

Dans le deuxième modèle, la linéarisation porte non plus sur un flux total, mais sur une différence de flux :

$$\sigma^t_{a,i,o} = \sum_{i'\in I}\sum_{o'\in\mathcal{O}} \left(f^a_{1,R^t_a,o',o} - f^a_{0,R^t_a,o',o}\right) u^{t-1}_{a,i',o'}. \tag{14}$$

L'objet à borner est donc plus fin. Au lieu d'utiliser une constante unique $M^{(2)}_a$, nous encadrons directement ce terme par deux bornes :

$$L^t_{a,i,o} \le \sigma^t_{a,i,o} \le U^t_{a,i,o}.$$

Comme $\sigma^t_{a,i,o}$ est une différence de contributions, ses amplitudes positive et négative sont en pratique plus faibles que celles d'un flux absolu. De plus, l'utilisation de bornes asymétriques permet de mieux épouser la vraie plage de variation de la variable. On obtient ainsi une formulation plus serrée que celle reposant sur un Big-M symétrique, ce qui améliore la relaxation linéaire et les performances du solveur.

3.4 Comparaison des deux modèles

Afin de comparer les deux modèles, nous les avons testés sur des instances aléatoires de 500 zones. La comparaison porte sur trois axes : le temps d'exécution, le nombre de variables et le nombre de contraintes. Le solveur **Gurobi** a été choisi pour sa performance et sa capacité à traiter efficacement des instances de grande taille dans des délais restreints. Les figures ci-dessous présentent les résultats obtenus.

Table 1 – Performances par instance à 500 zones

Seed	Temps de calcul (s)		Nombre de variables		Nombre de contraintes		Valeur objective		Relaxation continue	
	MILP initial	MILP reformulé	MILP initial	MILP reformulé	MILP initial	MILP reformulé	MILP initial	MILP reformulé	MILP initial	MILP reformulé
2	270	100	2 694 840	2 214 840	1 653 392	399 398	572 183.90	572 183.90	572 423.4	576 052.6
3	280	100	2 782 014	2 302 014	1 653 795	399 801	470 081.74	470 083.03	470 229.1	473 524.2
4	290	70	2 733 918	2 253 918	1 653 433	399 439	471 823.02	471 822.20	471 823.0	474 372.0
5	270	100	2 760 972	2 280 972	1 653 754	399 760	489 148.01	489 148.01	489 148.0	493 433.4
6	270	90	2 682 816	2 202 816	1 653 300	399 306	477 215.28	477 215.28	477 215.3	481 025.4
7	290	100	2 772 996	2 292 996	1 653 780	399 786	477 864.72	477 864.72	477 866.0	480 761.0
8	270	110	2 727 906	2 247 906	1 653 501	399 507	483 947.99	483 964.09	484 003.1	487 662.0
9	260	107	2 782 014	2 302 014	1 653 837	399 843	472 326.02	472 302.29	472 326.0	475 593.2
10	200	75	2 772 996	2 292 996	1 653 672	399 678	480 421.72	480 421.72	480 421.7	484 340.7
11	232	111	2 733 918	2 253 918	1 653 499	399 505	477 216.26	477 216.26	477 216.3	480 949.0
12	253	100	2 766 984	2 286 984	1 653 740	399 746	476 023.27	476 023.27	476 023.3	479 604.7
13	258	94	2 736 924	2 256 924	1 653 600	399 606	478 500.40	478 500.40	478 500.4	481 215.0
14	262	103	2 691 834	2 211 834	1 653 387	399 393	487 003.02	487 003.02	487 003.0	490 777.3
Moyenne	**265.0**	**96.9**	**2 753 395**	**2 273 395**	**1 653 574**	**399 580**	**479 504.26**	**479 504.55**	**479 630.7**	**483 024.0**

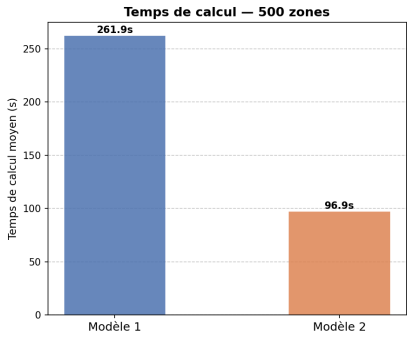


Figure 1 – Temps de calcul

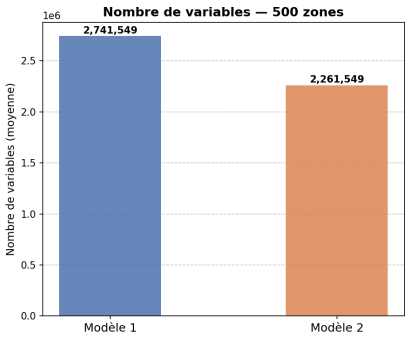


Figure 2 – Nombre de variables

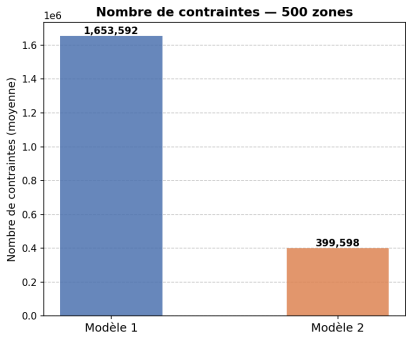


Figure 3 – Nombre de contraintes

On remarque que le MILP reformulé est significativement plus rapide, avec un temps d'exécution moyen de 96.9 s contre 265.0 s pour le MILP initial (–63%). Cette réduction s'explique principalement par la diminution du nombre de contraintes (–76%, de 1 653 574 à 399 580), le nombre de variables diminuant plus modérément (–17%, de 2 753 395 à 2 273 395). Les valeurs objectives restent quasi-identiques entre les deux formulations, confirmant que la reformulation ne dégrade pas la qualité des solutions. On note toutefois que la relaxation continue du MILP reformulé est moins serrée.

4 ALGORITHMES HEURISTIQUES ET MÉTAHEURISTIQUES

4.1 Motivations et positionnement

Le MILP^{reformulé} permet de donner une solution exact du problème c'est à dire la stratégie qui permet de maximiser le nombre de clients de la nouvelle génération de technologie. Cependant, le temps de calcul nécessaire à la résolution numérique du problème sur de vraies données peut être très long. En France, par exemple, le nombre total de sites dépasse les 71.000 sites[5].

Nous avons alors pensé à développer des heuristiques qui permettront de donner des stratégies qui s'approchent de l'optimal avec un temps d'éxécution moindre.

4.2 Fix-and-Relax

Principe du Fix-and-Relax Le Fix-and-Relax permet de réduire le nombre de variables à relaxer. C'est une heuristique de résolution itérative qui décompose un problème d'optimisation complexe sur un horizon temporel. À chaque étape, il définit une fenêtre glissante où les variables de décision conservent leur caractère entier (pour la précision), tandis que les variables futures sont relaxées en valeurs continues (pour simplifier les calculs) et les variables passées sont fixées (pour la stabilité). Une fois le sous-problème résolu, l'algorithme verrouille les décisions du cycle actuel, propage les activations si nécessaire, et avance à l'étape suivante jusqu'à ce que l'ensemble de l'horizon soit traité, permettant ainsi d'obtenir une solution de bonne qualité bien plus rapidement qu'une résolution globale.

Résultats de l'heuristique : Cette heuristique a été testé sur ces instances avec $W = 1$ (la fenêtre glissante) :

Table 2 – Temps de calcul et valeurs de la fonction objectif pour 5 instances à 150 zones

Modèle	**Seed 0**	**Seed 1**	**Seed 2**	**Seed 3**	**Seed 4**
MILP	1m 18.7s	2m 32.6s	12m 15.0s	9m 26.0s	58.1s
(solveur=highs)	117328.6109	110867.10	100512.42	115032.65	104383.65

Nous avons opté pour le solveur $HiGHS$, une solution open-source (non commerciale). Bien que ses temps de résolution sur nos instances soient supérieurs à ceux de *Gurobi*, ce choix permet d'évaluer de manière plus contrastée le gain de temps apporté par notre méthode.

Le temps de calcul sur ces instances a dépassé les 30 minutes, sans même aboutir à une solution. L'efficacité de la résolution du modèle complet repose en grande partie sur la phase du presolve du solveur $HiGHS$, qui exploite la structure globale du problème pour effectuer des déductions logiques fortes. Par exemple, dans le modèle intégral, la croissance des z_s^t permettent au solveur de propager l'intégralité d'un bout à l'autre de l'horizon de temps. Cette vision globale permet également d'éliminer les branches infaisables avant même le lancement de l'algorithme de Branch&Bound.

La méthode Fix-and-Relax brise cette chaîne de déductions quand elle relâche le futur dans le domaine continue. Les variables de décision binaires z_s^t par exemple perdent leur nature discrète pour devenir des variables continues. Par conséquent, la contrainte de liaison $z_s^{t+1} \geq z_s^t$ perd tout son pouvoir de coupe. Privé de cette

simplification mathématique, le solveur se retrouve contraint d'explorer "à l'aveugle" un arbre de recherche combinatoire gigantesque pour justifier chaque affectation de la fenêtre courante, ce qui provoque une explosion du temps de calcul (au-delà de 20 minutes) paradoxalement bien supérieure à celle du modèle global.

4.3 Algorithme génétique initial basé sur un solveur

Rôle et inspiration de l'algorithme génétique. Les algorithmes génétiques constituent une famille classique de métaheuristiques inspirées des mécanismes de sélection, croisement et mutation [9]. Nous adaptons cette approche au problème étudié en représentant chaque individu par les dates de déploiement des sites.

Il est particulièrement adapté aux problèmes d'optimisation combinatoire de grande taille, pour lesquels les méthodes exactes deviennent rapidement intractables.

Dans notre contexte, le problème de planification du déploiement du réseau NG présente un espace de recherche exponentiel en le nombre de sites et de périodes, rendant l'utilisation directe d'un solveur MILP coûteuse en temps de calcul. L'AG permet alors d'explorer efficacement cet espace en maintenant une population de solutions candidates et en favorisant progressivement les individus les plus performants.

4.3.1 • Principe de l'algorithme

Table 3 – Notations algorithmiques utilisées dans l'AG direct

Symbole	Signification	Symbole	Signification
N	taille de la population	$\mathcal{P}$	population courante
G	nombre maximal de générations	$\mathcal{P}_{new}$	nouvelle population
E	nombre d'individus élites conservés	$f(ind)$	fitness
K	taille du tournoi	$ind^\star$	meilleur individu
μ	taux de mutation	$f^\star$	meilleure fitness
T_{lim}	limite de temps	z_s^t	état du site s à la date t
ind	individu (temps de déploiement des sites)	r_a^t	couverture de la zone a à la date t
$ind[k]$	date de déploiement du site s_k ou -1	$t_{start}(a)$	première date de couverture de a

Dans cette première version, l'algorithme génétique repose sur un encodage direct des variables de décision binaires du modèle. Chaque individu est ainsi représenté par un vecteur

$$ind = (z_s^t)_{s\in\mathcal{S},\, t\in\mathcal{T}} \in \{0,1\}^{|\mathcal{S}|\times|\mathcal{T}|},$$

où chaque gène indique si un site s est équipé de la technologie NG à la date t, i.e $z_s^t = 1$ indique que le site s est équipé de la technologie NG à la période t.

L'évaluation d'un individu consiste à considérer ce vecteur comme une affectation partielle des variables du problème, puis à résoudre le modèle d'optimisation conditionnel correspondant. Plus précisément, les variables z_s^t sont fixées, et le solveur détermine les variables restantes de manière à maximiser la fonction objectif tout en respectant les contraintes du modèle.

La fitness d'un individu est définie par :

$$f(ind) = \begin{cases} \max\left\{\text{objectif}(z,u,r) \mid z_s^t = ind_{s,t},\ (u,r) \text{ faisables}\right\}, & \text{si le problème est faisable,} \\ -\infty, & \text{sinon.} \end{cases}$$

Cette procédure revient à associer à chaque individu la valeur optimale du sous-problème induit par la fixation des variables z_s^t, ce qui permet de définir une fonction d'évaluation cohérente avec la formulation mathématique du problème.

Algorithm 1: Algorithme génétique direct

Input: N, G, μ, E, K, T_{lim}
Output: $ind^\star, f^\star$

$\mathcal{P} \leftarrow$ InitializePopulation() $ind^\star \leftarrow \varnothing, \quad f^\star \leftarrow -\infty$
for $g = 1$ **to** G **do**
  if $temps \geq T_{lim}$ **then**
    break
  Évaluer tous les individus de $\mathcal{P}$ via Evaluate `// cache + filtration`
  $ind_{best} \leftarrow \arg\max_{ind \in \mathcal{P}} f(ind)$
  if $f(ind_{best}) > f^\star$ **then**
    $ind^\star \leftarrow ind_{best}$; $\quad f^\star \leftarrow f(ind_{best})$
  $\mathcal{P}_{new} \leftarrow$ les E meilleurs individus `// élitisme`
  while $|\mathcal{P}_{new}| < N$ **do**
    $p_1, p_2 \leftarrow$ TournamentSelection$(\mathcal{P}, K)$
    $c \leftarrow$ Crossover(p_1, p_2)
    $c \leftarrow$ Mutate(c, μ)
    $c \leftarrow$ Repair(c)
    Ajouter c à $\mathcal{P}_{new}$
  $\mathcal{P} \leftarrow \mathcal{P}_{new}$
return $(ind^\star, f^\star)$

InitializePopulation :

L'initialisation aléatoire d'une population à N individus suivi par un passage dans la fonction Repair.

Evaluate :

L'évaluation d'un individu *ind* consiste à fixer les variables z_s^t dans le modèle d'optimisation, puis à résoudre le problème associé à l'aide d'un solveur MILP. Afin de réduire le nombre d'appels au solveur, une mémoïsation est utilisée : si *ind* a déjà été évalué, sa fitness est directement récupérée depuis un cache.

Une vérification préalable permet d'écarter rapidement les individus violant les contraintes de déploiement Z^t, auxquels une pénalité (très faible valeur) est attribuée. Si le problème est faisable, la fitness correspond à la valeur optimale de la fonction objectif ; sinon, une pénalité est appliquée.

Opérateurs génétiques :

Les opérateurs génétiques assurent l'exploration de l'espace des solutions en combinant et modifiant les individus au cours des générations. Ils reposent sur trois mécanismes principaux : sélection, croisement et mutation.

Sélection. La sélection par tournoi consiste à choisir un individu parmi un sous-ensemble aléatoire de la population, favorisant ainsi les individus de meilleure fitness.

Algorithm 2: Sélection par tournoi

Input: Population $\mathcal{P}$, fitness associées, taille de tournoi K
Output: Un individu sélectionné
Tirer uniformément K individus distincts de $\mathcal{P}$
return *l'individu de meilleure fitness parmi ces K candidats*

Croisement. Le croisement permet de générer de nouveaux individus en combinant deux solutions parentales. Dans l'implémentation considérée, il s'agit d'un croisement à un point sur la représentation vectorielle.

Algorithm 3: Crossover (croisement à un point)

Input: Deux parents p_1, p_2
Output: Deux enfants c_1, c_2
Tirer un point de coupure k uniformément
$c_1 \leftarrow p_1[1:k] \parallel p_2[k+1:\text{fin}]$
$c_2 \leftarrow p_2[1:k] \parallel p_1[k+1:\text{fin}]$
Appliquer Repair à c_1 et c_2
return c_1, c_2

Mutation. La mutation introduit de la diversité en modifiant aléatoirement certains gènes d'un individu.

Algorithm 4: Mutation

Input: Individu *ind*, taux de mutation μ
Output: Individu muté
for *chaque gène j de ind* **do**
  Avec probabilité μ, inverser la valeur du gène j
Appliquer Repair à *ind*
return *ind*

Réparation. Après croisement et mutation, une étape de réparation est appliquée afin de garantir la faisabilité des individus vis-à-vis des contraintes de déploiement.

Algorithm 5: Réparation (version simplifiée)

Input: Individu *ind*
Output: Individu faisable
Ajuster les dates de déploiement afin de satisfaire $\forall t,\ n_t(ind) \leq Z^t$
Réaffecter les sites en excès vers des périodes admissibles si possible
return *ind*

4.3.2 • Complexité.

Soient N la taille de la population, G le nombre de générations, $|\mathcal{S}|$ le nombre de sites et $|\mathcal{T}|$ le nombre de périodes. La taille d'un individu est de l'ordre de $|\mathcal{S}| \times |\mathcal{T}|$.

Le coût dominant de l'algorithme provient de l'évaluation des individus, qui nécessite la résolution d'un problème MILP. Dans le pire cas, le solveur est appelé $N \times G$ fois, ce qui conduit à une complexité globale de la forme :

$$\mathcal{O}\big(N \times G \times T_{\text{solve}}\big),$$

où T_{solve} désigne le temps de résolution du modèle.

Les opérations génétiques (sélection, croisement, mutation, réparation) ont un coût polynomial en la taille des individus, typiquement en $\mathcal{O}(|\mathcal{S}| \times |\mathcal{T}|)$, et sont négligeables devant le coût des appels au solveur.

Ainsi, la performance de cette approche est principalement limitée par le nombre de résolutions du modèle, ce qui motive le recours à des méthodes d'évaluation plus rapides.

4.3.3 • Résultats et analyse

Les tests de l'algorithme génétique, effectués sur les mêmes instances que le Fix-and-Relax ($N = 30, G = 15, E = 5, \mu = 0,01$), révèlent des performances critiques. Dès la première génération, les temps de calcul augmentent de manière exponentielle, empêchant fréquemment l'obtention d'un résultat. Alors que le solveur $HiGHS$ parvient à résoudre ces instances plus rapidement, l'algorithme génétique dépasse systématiquement la limite de temps de 30 minutes.

4.3.4 • Limites de cette approche.

Cette première version présente un inconvénient majeur : l'évaluation de chaque individu nécessite la résolution complète du modèle d'optimisation, ce qui entraîne des temps de calcul très élevés.

En effet, pour une population de taille N et G générations, le solveur est appelé jusqu'à $N \times G$ fois, ce qui devient rapidement intractable pour des instances réalistes.

Cette limitation nous a conduits à proposer une seconde approche, basée sur un encodage direct des temps de déploiement et une évaluation sans solveur, présentée dans la section suivante.

4.4 Algorithme génétique direct avec initialisation gloutonne structurée

Évolution par rapport à l'AG initial. La version précédente de l'algorithme génétique présentait une limite majeure : la fitness d'un individu nécessitait la résolution d'un sous-problème MILP, ce qui rendait chaque évaluation coûteuse. La version suivante conserve la même logique évolutionnaire générale (population, sélection, croisement, mutation, réparation), mais introduit trois changements structurants :

1. un **nouvel encodage** de l'individu, fondé sur les dates de premier déploiement ;
2. une **évaluation directe sans solveur**, obtenue par reconstruction analytique des variables dérivées ;
3. une **initialisation gloutonne enrichie**, permettant de démarrer la recherche dans une région prometteuse de l'espace des solutions.

L'objectif n'est donc plus de modifier l'ossature de l'AG, mais d'en réduire drastiquement le coût d'évaluation et d'améliorer la qualité de la population initiale.

4.4.1 • Nouveau codage des individus

Au lieu de représenter explicitement toutes les variables binaires z_s^t, chaque individu est désormais codé par un vecteur de taille $|\mathcal{S}|$:

$$ind = \big(ind(s)\big)_{s\in\mathcal{S}}, \qquad ind(s) \in \mathcal{T} \cup \{-1\},$$

où $ind(s)$ désigne la date de premier déploiement du site s, et -1 signifie que le site n'est jamais activé.

Les variables binaires du modèle se reconstruisent alors par :

$$z_s^t = \begin{cases} 1, & \text{si } ind(s) \neq -1 \text{ et } t \geq ind(s), \\ 0, & \text{sinon.} \end{cases}$$

Cette représentation est plus compacte et incorpore naturellement la monotonie des déploiements. Elle permet également de raisonner directement sur les *dates d'activation* des sites, ce qui rend les opérateurs de voisinage plus interprétables.

4.4.2 • Évaluation directe de la fitness

Le changement principal concerne l'évaluation. Une fois les dates de déploiement fixées, les variables de couverture r_a^t puis les flux d'utilisateurs $u_{a,i,o}^t$ peuvent être recalculés directement, sans relancer de solveur.

On commence par reconstruire :

$$r_a^t = \mathbf{1}\left[\exists s \in \mathcal{S}_a \ : \ z_s^t = 1\right].$$

Autrement dit, dès que les variables z_s^t sont fixées, les variables de couverture r_a^t deviennent elles aussi déterministes.

Récurrence directe sur les utilisateurs. Soit τ l'opérateur étudié (ici ORANGE), et soit $NG = \text{o5G-ORANGE}$ l'offre cible. Pour chaque zone a, chaque période t, chaque opérateur i et chaque offre o, les variables $u_{a,i,o}^t$ sont calculées à partir des valeurs à la période précédente.

On note :

$$C_{t,a}^{(1)} = \big(1,\ R_{t,a,i_2}^{comp}, \ldots, R_{t,a,i_{|I|}}^{comp}\big), \qquad C_{t,a}^{(0)} = \big(0,\ R_{t,a,i_2}^{comp}, \ldots, R_{t,a,i_{|I|}}^{comp}\big),$$

c'est-à-dire le vecteur de couverture concurrentielle complété par la décision locale $r_a^t = 1$ ou $r_a^t = 0$.

La dynamique observée dans le code s'écrit alors :

$$u_{0,a,i,o} = u_{a,i,o}^0,$$

et, pour tout $t \geq 1$,

$$u_{t,a,i,o} = r_a^t \sum_{i' \in I} \sum_{o' \in \mathcal{O}_{i'}} f_{a,C_{t,a}^{(1)},o',o}\ u_{t-1,a,i',o'} \ + \ (1 - r_a^t) \sum_{i' \in I} \sum_{o' \in \mathcal{O}_{i'}} f_{a,C_{t,a}^{(0)},o',o}\ u_{t-1,a,i',o'}.$$

Par conséquent, lorsque z est fixé, toutes les variables restantes du modèle se déduisent successivement :

$$z \implies r \implies u.$$

L'objectif final s'obtient alors directement à l'horizon terminal :

$$f(ind) = \sum_{a \in \mathcal{A}} u_{T_{\max},a,\tau,NG}.$$

La fitness correspond donc au nombre final d'utilisateurs ORANGE en offre NG, obtenu sans résolution d'un problème d'optimisation auxiliaire. Ainsi, l'évaluation n'est plus une résolution d'optimisation conditionnelle, mais une suite de calculs déterministes. Ce changement transforme le coût d'une évaluation, qui passe d'un appel solveur à un calcul direct en temps polynomial.

Précalcul des trajectoires. Afin d'accélérer davantage l'évaluation, on exploite le fait que, pour une zone a donnée, la dynamique des utilisateurs dépend uniquement de la décision de couverture binaire r_a^t au cours du temps. En pratique, cette dynamique est entièrement déterminée par la date de première couverture :

$$t_{start}(a) = \min\{\, t \in \mathcal{T} \mid r_a^t = 1 \,\},$$

avec la convention $t_{start}(a) = +\infty$ si la zone n'est jamais couverte.

On effectue donc, pour chaque zone a et pour chaque valeur possible de cette décision, un précalcul unique de la trajectoire associée des utilisateurs ORANGE en offre NG. Plus précisément, pour chaque $t_{start}(a)$ admissible, on calcule une fois pour toutes la suite :

$$\left(u_{t,a,\tau,NG}^{(t_{start})}\right)_{t \in \mathcal{T}},$$

puis on la mémorise.

Lors de l'évaluation d'un individu, il suffit d'identifier la valeur de $t_{start}(a)$ induite par la solution et de relire la trajectoire déjà calculée. Le principe est donc simple : *une décision de couverture donnée est évaluée une seule fois, puis mémorisée pour toutes les évaluations futures.*

Vérification manuelle des contraintes. Dans cette version, certaines contraintes sont contrôlées manuellement après reconstruction de z, r et u.

Contrainte de déploiement. Pour toute période t, on note le nombre de nouveaux sites déployés :

$$n_t(ind) = \begin{cases} \sum_{s\in\mathcal{S}_\tau} z_s^t, & \text{si } t = \min(\mathcal{T}), \\ \sum_{s\in\mathcal{S}_\tau} \left(z_s^t - z_s^{t-1}\right), & \text{sinon.} \end{cases}$$

On impose alors :

$$n_t(ind) \le \overline{Z}^t, \qquad \forall t \in \mathcal{T}.$$

Contrainte de capacité NG globale. Le total des utilisateurs de l'offre NO_τ à la période t est

$$U_t^{NO_\tau} = \sum_{a\in\mathcal{A}} u_{a,\tau,NO_\tau}^t.$$

La demande totale $D_{NG}^t U_t^{NO_\tau}$ ne doit pas dépasser la capacité totale installée :

$$D_{NG}^t \sum_{a\in\mathcal{A}} u_{a,\tau,NO_\tau}^t \le CAPA_{NG}^t \sum_{s\in\mathcal{S}_\tau} z_s^t, \qquad \forall t \in \mathcal{T}.$$

Contrainte de capacité par site exclusif. On contrôle aussi les utilisateurs de NO_τ qui ne peuvent être servis que par un seul site actif s. On définit :

$$\mathcal{A}_{t,s}^{exc} = \left\{ a \in \mathcal{A}_s \;\middle|\; z_{s'}^t = 0,\ \forall s' \in \mathcal{S}_{a,\tau} \setminus \{s\} \right\}.$$

La charge exclusive du site s est alors :

$$U_{t,s}^{exc} = \sum_{a\in\mathcal{A}_{t,s}^{exc}} u_{a,\tau,NO_\tau}^t.$$

Elle doit respecter la capacité individuelle du site :

$$U_{t,s}^{exc} \le \frac{CAPA_{NG}^t}{D_{NG}^t}, \qquad \forall t \in \mathcal{T},\ \forall s \in \mathcal{S}_\tau \text{ tel que } z_s^t = 1.$$

Contrainte de couverture minimale. Enfin, la population couverte doit atteindre le seuil stratégique QA^t :

$$\sum_{a\in\mathcal{A}} u_a\, r_a^t \ge QA^t \sum_{a\in\mathcal{A}} u_a, \qquad \forall t \in \mathcal{T}.$$

Remarque de formulation. On peut interpréter cette étape comme la résolution d'un sous-problème *analytique* : une fois z fixé, les variables r et u ne sont plus des variables d'optimisation, mais des variables dérivées définies explicitement par les relations précédentes. L'algorithme remplace donc un sous-problème MILP par une procédure déterministe de reconstruction suivie d'un test de faisabilité.

Filtration et mémoïsation. Cette version introduit également deux mécanismes de réduction du coût :

— une **filtration précoce** des individus violant immédiatement la contrainte de déploiement Z^t ;

— un **cache de fitness**, évitant de recalculer des individus déjà évalués.

Ainsi, un individu déjà rencontré retourne instantanément sa valeur de fitness, et un individu manifestement infaisable reçoit directement une pénalité.

4.4.3 • Initialisation gloutonne structurée

Contrairement à l'initialisation purement aléatoire de l'AG initial, cette version construit la population autour d'une solution gloutonne faisable.

Chaque site s reçoit un score d'importance fondé sur la population totale des zones qu'il peut couvrir :

$$\text{score}(s) = \sum_{a \in \mathcal{A}_s} u_a.$$

Les sites sont ensuite triés par score décroissant, puis déployés le plus tôt possible en respectant les capacités temporelles Z^t. On obtient ainsi un individu glouton de référence, noté ind^g.

La population initiale n'est pas constituée de simples copies de cette solution, mais de plusieurs variantes construites autour de celle-ci.

Table 4 – Composition de la population initiale dans l'AG direct

Catégorie	Rôle
Glouton pur	solution de référence ind^g
Greedy swap-k	remplace certains sites déployés par de meilleurs sites non déployés
Greedy time-shift-k	retarde le déploiement de certains sites déjà activés
Greedy jitter	conserve l'idée gloutonne avec ordre partiellement aléatoire
Individus aléatoires	diversification globale

Voisinage *greedy swap*. L'opérateur *swap* part de l'individu glouton ind^g et remplace les sites déployés les moins bien classés par les meilleurs sites non encore retenus, tout en conservant les dates de déploiement. Il permet d'explorer des combinaisons proches du glouton, sans s'en éloigner brutalement. Autrement dit, on échange des sites peu prioritaires déjà sélectionnés contre des sites mieux classés restés inactifs.

Voisinage *greedy time-shift*. L'opérateur *time-shift* conserve les mêmes sites, mais retarde le déploiement de certains d'entre eux d'une période. Il explore ainsi des plans plus prudents temporellement, tout en restant dans le voisinage immédiat de la solution gloutonne.

Voisinage *greedy jitter*. L'opérateur *jitter* conserve la logique gloutonne de sélection des sites, mais introduit une perturbation légère en randomisant partiellement l'ordre de parcours. Il produit ainsi des individus qui restent proches du schéma glouton tout en apportant davantage de diversité à la population initiale.

Algorithm 6: Initialisation structurée de la population

Input: Taille N
Output: Population initiale $\mathcal{P}$

Construire un individu glouton faisable ind^g
$\mathcal{P} \leftarrow \{ind^g\}$
Ajouter $\lfloor N/5 \rfloor$ variantes GreedySwap(ind^g, k)
Ajouter $\lfloor N/5 \rfloor$ variantes GreedyTimeShift(ind^g, k)
Ajouter $\lfloor N/5 \rfloor$ variantes GreedyJitter(ind^g)
Compléter avec des individus aléatoires réparés jusqu'à atteindre N
return $\mathcal{P}$

4.4.4 • Boucle génétique inchangée, mais plus efficace

La structure générale de l'algorithme reste identique à celle de l'AG initial : élitisme, sélection par tournoi, croisement, mutation puis réparation. La différence provient du fait que l'évaluation est désormais très rapide, ce qui permet de traiter des populations plus riches et un nombre de générations bien plus important dans un même budget de temps.

Algorithm 7: Algorithme génétique direct avec initialisation gloutonne

Input: N, G, μ, E, K, T_{lim}
Output: $ind^\star, f^\star$

$\mathcal{P} \leftarrow$ StructuredInitializePopulation(N)
$ind^\star \leftarrow \varnothing, \quad f^\star \leftarrow -\infty$
for $g = 1$ **to** G **do**
  if $temps \geq T_{lim}$ **then**
    break
  Évaluer tous les individus de $\mathcal{P}$ via EvaluateDirect
  Mettre à jour $(ind^\star, f^\star)$
  $\mathcal{P}_{new} \leftarrow$ les E meilleurs individus
  while $|\mathcal{P}_{new}| < N$ **do**
    $p_1, p_2 \leftarrow$ TournamentSelection$(\mathcal{P}, K)$
    $c \leftarrow$ Crossover(p_1, p_2)
    $c \leftarrow$ Mutate(c, μ)
    $c \leftarrow$ Repair(c)
    Ajouter c à $\mathcal{P}_{new}$
  $\mathcal{P} \leftarrow \mathcal{P}_{new}$
return $(ind^\star, f^\star)$

4.4.5 • Complexité

On note G le nombre de generations, S le nombre de sites et A le nombre d'areas.

Apres le pre-calcul, l'evaluation d'un individu parcourt les sites et les areas pour reconstruire les deploiements, les couvertures et verifier les contraintes. En ne gardant que les dimensions dominantes, le cout d'une evaluation est :$O(SA)$.

A chaque generation, l'algorithme evalue une population de taille fixe. Comme la taille du population est un parametre constant dans nos experiences, le cout d'une generation reste :$O(SA)$.

Ainsi, pour G generations, la complexite de la version AG direct est :$O(GSA)$. En ajoutant le pre-calcul, la complexite totale devient :$O(S + A + GSA)$. Comme GSA domine $S + A$, on retient :$O(GSA)$.

En prenant $G = O(A)$ (cas pratique), on obtient : $O(SA^2)$.

Bilan. Cette version intermédiaire constitue une étape décisive entre l'AG initial basé sur solveur et l'algorithme mémétique. Elle n'introduit pas encore de recherche locale explicite, mais elle transforme profondément l'efficacité de l'approche grâce à l'encodage par dates de déploiement, à l'évaluation directe, au précalcul, au cache, à la filtration et à l'initialisation gloutonne structurée.

4.4.6 • Résultats et analyse

L'algorithme a été testé sur plusieurs tailles d'instances. Le paramètre n_a désigne le nombre de zones sélectionnées aléatoirement dans les données. Dans cette première série d'expériences, nous considérons les instances suivantes :

$$n_a \in \{200, 350, 550\}.$$

Les paramètres de l'algorithme génétique direct sont les suivants :

Table 5 – Paramètres utilisés pour l'AG direct

Paramètre	**Valeur**
Taille de population	100
Nombre de générations	n_a
Taux de mutation	0.05
Nombre d'élites	8

Remarque sur le solveur. Le solveur GLPK a été utilisé dans un premier temps pour des raisons de simplicité sur de petites instances. Cependant, il est rapidement dominé par des solveurs plus performants tels que `HiGHS`, tant en temps de calcul qu'en qualité de solution. Les expériences principales s'appuient donc sur ces solveurs plus avancés, afin d'obtenir des comparaisons plus représentatives.

Les résultats obtenus sont présentés dans le tableau suivant.

Table 6 – Temps de calcul et valeurs de la fonction objectif pour l'AG direct

Instance	$n_a = 200$	$n_a = 350$	$n_a = 550$
Solution exacte avec appsi highs	170s	1156.7s	2773.3s
initiale	149073.5804	313155.3180	536153.70665
AG direct	40.69s	125.91s	341.14s
avec glouton structuré	107734.9266	256859.0544	437422.0351

Pour mieux visualiser le comportement de l'algorithme, nous avons également tracé l'évolution de la meilleure valeur obtenue au cours des générations. Ces courbes permettent d'observer la rapidité de convergence de l'AG direct ainsi que les phases éventuelles de stagnation.

On remarque que cette nouvelle version permet de traiter des instances beaucoup plus grandes que la première version de l'AG. En effet, l'évaluation directe évite les appels répétés au solveur, ce qui réduit fortement le temps de calcul. De plus, l'initialisation gloutonne permet de partir d'une solution faisable de bonne qualité, ce qui améliore la valeur obtenue dès les premières générations.

Cependant, les courbes montrent également que l'algorithme peut atteindre assez rapidement un palier. Après les premières améliorations, les opérateurs classiques de croisement et de mutation ne parviennent pas toujours à améliorer davantage la meilleure solution. Cela montre que l'AG direct explore efficacement l'espace des solutions, mais qu'il peut manquer d'une phase d'amélioration locale autour des meilleurs individus.

Cette limite motive l'évolution vers l'algorithme mémétique direct, qui conserve la même évaluation rapide sans solveur, mais ajoute une recherche locale afin d'améliorer plus finement les meilleures solutions obtenues.

4.5 Algorithme mémétique direct

Évolution par rapport à l'AG direct : L'algorithme mémétique direct reprend l'ensemble des briques de l'AG direct : même codage par dates de déploiement, même évaluation directe, mêmes précalculs, même

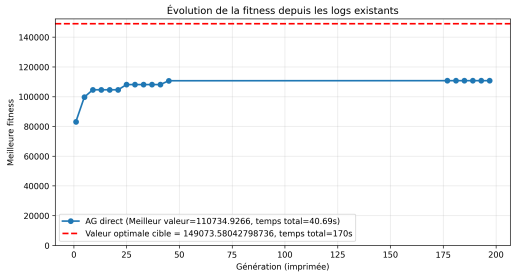


Figure 4 – AG direct pour $n_a = 200$

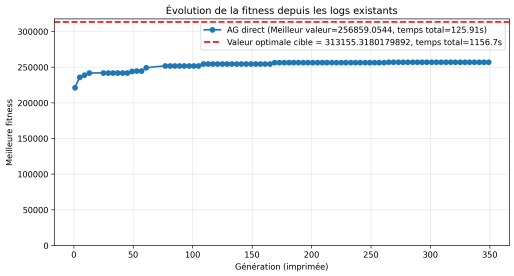


Figure 5 – AG direct pour $n_a = 350$

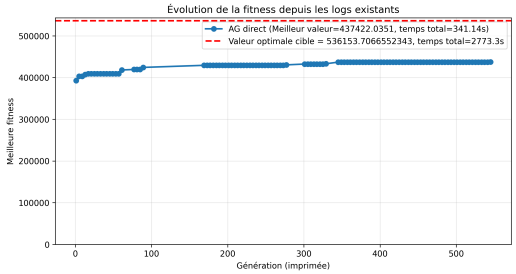


Figure 6 – AG direct pour $n_a = 550$

filtration, même réparation et même structure générale de population. La différence essentielle est l'ajout d'une phase d'**intensification locale** au sein de chaque génération [12, 13].

4.5.1 • Principe général

À chaque génération, l'algorithme procède comme l'AG direct, mais ajoute une étape supplémentaire : une recherche locale de type *first-improvement* est appliquée au meilleur individu de la population avant la phase de reproduction.

Cette recherche locale repose sur des mouvements élémentaires directement interprétables dans le codage choisi :

— avancer le déploiement d'un site : $ind(s) \leftarrow ind(s) - 1$;
— retarder le déploiement d'un site : $ind(s) \leftarrow ind(s) + 1$;
— désactiver un site : $ind(s) \leftarrow -1$;
— pour un site non déployé, tenter une activation sur les premières périodes admissibles.

Chaque voisin généré est réparé puis évalué. Dès qu'un voisin strictement améliorant est trouvé, il est accepté, et l'algorithme passe au site suivant.

4.5.2 • Recherche locale *first-improvement*

Ce choix est cohérent avec le cadre de l'AG direct : puisque l'évaluation est rapide, on peut explorer plusieurs voisins sans coût prohibitif, tout en évitant une recherche locale trop lourde.

Algorithm 8: Recherche locale *first-improvement*

Input: Individu ind, fitness courante $f(ind)$, budget L
Output: Individu amélioré ind^{loc}, fitness f^{loc}

$ind^{loc} \leftarrow ind, \quad f^{loc} \leftarrow f(ind)$
$evals \leftarrow 0$
Parcourir les sites dans un ordre aléatoire
for *chaque site* s **do**
  if $evals \geq L$ **then**
    break
  Construire une liste de mouvements candidats autour de $ind(s)$
  for *chaque mouvement candidat* **do**
    Générer un voisin ind' en modifiant uniquement le site s
    $ind' \leftarrow$ Repair(ind')
    Calculer $f(ind')$
    $evals \leftarrow evals + 1$
    if $f(ind') > f^{loc}$ **then**
      $ind^{loc} \leftarrow ind'$
      $f^{loc} \leftarrow f(ind')$
      break
return (ind^{loc}, f^{loc})

4.5.3 • Mutations structurées

La seconde différence importante avec l'AG direct concerne l'opérateur de mutation. Dans la version mémétique, la mutation n'est plus uniquement aléatoire. Elle devient **structurée**, c'est-à-dire orientée par des transformations pertinentes au regard de la sémantique du codage.

Les principales mutations utilisées sont les suivantes :

— ***swap_periods*** : échange les dates de déploiement de deux sites déjà activés ;
— ***advance_best*** : active l'un des meilleurs sites non déployés sur la période admissible la plus précoce ;
— ***delay_worst*** : retarde d'une période le site déployé le moins bien classé ;
— **mutation aléatoire résiduelle** : modifie certains gènes au hasard, puis applique Repair.

Dans l'implémentation étudiée, ces mutations sont sélectionnées selon les proportions suivantes :

$$33\% \text{ pour } swap_periods, \qquad 22\% \text{ pour } advance_best, \qquad 15\% \text{ pour } delay_worst,$$

et le cas restant correspond à une mutation aléatoire classique. Ces transformations permettent d'introduire de la diversité tout en restant proches de la structure du problème.

Algorithm 9: Mutation structurée

Input: Individu ind, taux de mutation μ
Output: Individu muté

Tirer un réel x uniformément dans $[0, 1]$
if $x < 0.33$ **then**
  appliquer SwapPeriods(ind)
else
  if $x < 0.55$ **then**
    appliquer AdvanceBest(ind)
  else
    if $x < 0.70$ **then**
      appliquer DelayWorst(ind)
    else
      appliquer une mutation aléatoire de taux μ
      appliquer Repair
return ind

4.5.4 • Boucle mémétique

La boucle principale reste proche de celle de l'AG direct. La seule différence structurelle est l'insertion, à chaque génération, d'une recherche locale sur le meilleur individu courant avant la création de la nouvelle population.

Algorithm 10: Algorithme mémétique direct

Input: $N, G, \mu, E, K, L, T_{lim}$
Output: $ind^\star, f^\star$

$\mathcal{P} \leftarrow$ StructuredInitializePopulation(N)
$ind^\star \leftarrow \varnothing, \quad f^\star \leftarrow -\infty$
for $g = 1$ **to** G **do**
  if $temps \geq T_{lim}$ **then**
    break
  Évaluer tous les individus de $\mathcal{P}$ via EvaluateDirect
  $ind_{best} \leftarrow \arg\max_{ind \in \mathcal{P}} f(ind)$
  $(ind_{loc}, f_{loc}) \leftarrow$ LocalSearch($ind_{best}, f(ind_{best}), L$)
  if $f_{loc} > f(ind_{best})$ **then**
    remplacer ind_{best} par ind_{loc} dans la population
  Mettre à jour $(ind^\star, f^\star)$
  $\mathcal{P}_{new} \leftarrow$ les E meilleurs individus
  while $|\mathcal{P}_{new}| < N$ **do**
    $p_1, p_2 \leftarrow$ TournamentSelection($\mathcal{P}, K$)
    $c \leftarrow$ Crossover(p_1, p_2)
    $c \leftarrow$ StructuredMutate(c, μ)
    Ajouter c à $\mathcal{P}_{new}$
  $\mathcal{P} \leftarrow \mathcal{P}_{new}$
return $(ind^\star, f^\star)$

4.5.5 • Complexité

La version memetique reprend la meme evaluation directe que l'AG classique, donc le cout de base par generation est aussi $O(SA)$. Elle ajoute une recherche locale sur le meilleur individu de chaque generation.

Cette recherche locale teste un nombre fixe de voisins, note `ls_neighbors`. Comme ce nombre est fixe dans l'implementation, il ne change pas l'ordre asymptotique. Chaque voisin est evalue avec le meme cout $O(SA)$, donc le cout par generation reste asymptotiquement $O(SA)$. Pour G generations, la complexite de la version memetique (le precacul est negligeable devant ça) est $O(GSA)$. En prenant $G = O(A)$ (choix pratique), on trouve $O(SA^2)$.

La version memetique garde donc le meme ordre de complexite que l'AG direct, mais avec une constante plus grande a cause des evaluations supplementaires de la recherche locale.

4.5.6 • Résultats et analyse

L'algorithme mémétique direct a été testé sur les mêmes instances que l'AG direct, Le nombre de zones considérées sont donc :

$$n_a \in \{200, 350, 550, 600\}.$$

Table 7 – Paramètres utilisés pour l'algorithme mémétique direct

Paramètre	**Valeur**
Taille de population	100
Nombre de générations	n_a
Taux de mutation	0.05
Nombre d'élites	8
Nombre maximal de voisins testés	60

Les résultats obtenus sont présentés dans le tableau suivant.

Table 8 – Temps de calcul et valeurs de la fonction objectif pour l'algorithme mémétique direct

Instance	$n_a = 200$	$n_a = 350$	$n_a = 550$	$n_a = 600$
AG direct avec glouton structuré	41.51s 114564.938	124.70s 254192.942	320.81s 437422.0351	428.34s 494461.1975
Algorithme mémétique direct	58.35s 143726.911	192.56s 301622.452	469.62s 524719.8032	674.42s 586214.2419
Solution exacte avec appsi highs	170s 149073.5804	1156.7s 313155.3180	2773.3s 536153.7067	>1heure 604198.0103

Les figures suivantes présentent l'évolution de la meilleure fitness au cours des générations. Elles permettent de comparer directement l'AG direct et sa version mémétique.

4.5.7 • Conclusion sur l'évolution des heuristiques.

L'évolution progressive des heuristiques montre clairement l'intérêt de la démarche adoptée. La première version de l'algorithme génétique, basée sur des appels répétés au solveur, permettait de conserver une évaluation fidèle au modèle exact, mais devenait rapidement trop coûteuse dès que la taille des instances augmentait. L'AG direct a permis de lever ce verrou en remplaçant l'appel au solveur par une évaluation analytique rapide, fondée sur la reconstruction directe des variables z, r et u.

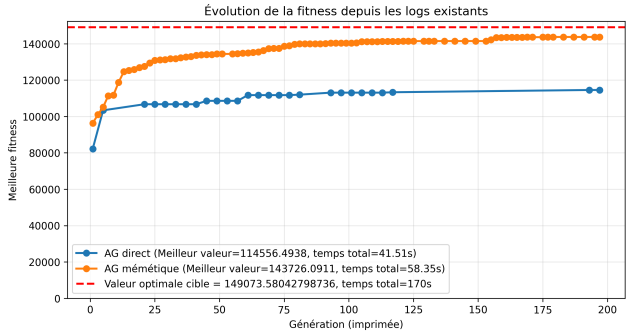


Figure 7 – Comparaison AG direct et mémétique pour $n_a = 200$

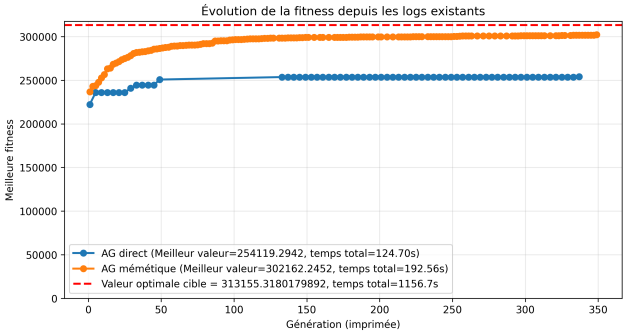


Figure 8 – Comparaison AG direct et mémétique pour $n_a = 350$

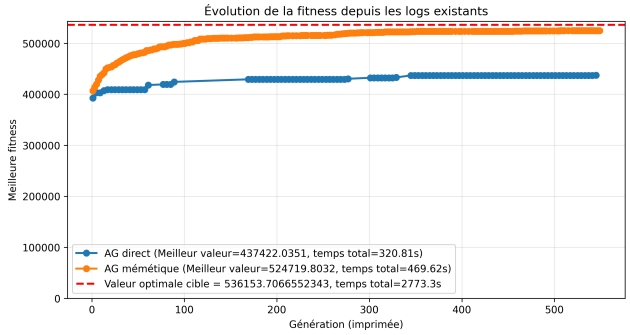


Figure 9 – Comparaison AG direct et mémétique pour $n_a = 550$

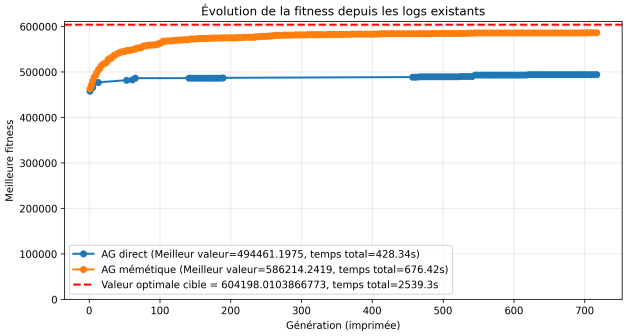


Figure 10 – Comparaison AG direct et mémétique pour $n_a = 600$

L'algorithme mémétique constitue l'étape suivante de cette évolution. Il conserve la rapidité de l'AG direct, tout en ajoutant une recherche locale capable d'améliorer les meilleures solutions obtenues. Les résultats montrent que cette intensification permet de se rapprocher davantage de la solution exacte, au prix d'un temps de calcul modérément supérieur.

Ainsi, l'intérêt principal de l'algorithme mémétique est d'offrir un compromis efficace entre qualité de solution et temps d'exécution. Il reste suffisamment rapide pour traiter des instances de grande taille, tout en produisant des plans de déploiement nettement meilleurs que ceux obtenus par l'AG direct seul. Cette approche apparaît donc comme la plus pertinente parmi les heuristiques testées pour traiter des instances réalistes, où la résolution exacte devient trop coûteuse.

5
MIXED-INTEGER LINEAR PROGRAMMING GRAPH NEURAL NETWORK (MILPGNN)

5.1 Motivations et positionnement

L'évolution des méthodes de résolution vers les algorithmes génétiques et mémétiques a permis d'obtenir des solutions de haute qualité en diminuant considérablement les temps de calcul. Toutefois, ces métaheuristiques reposent sur une exploration stochastique qui doit être réinitialisée pour chaque nouvelle instance. L'introduction de l'approche MILPGNN répond à une ambition différente : exploiter la topologie spécifique du réseau de sites via des réseaux de neurones de graphes (GNN) pour guider le solveur.

Cette méthode se positionne comme une stratégie de « démarrage à chaud » structurelle. En utilisant le GNN pour prédire un ratio de variables de décision (z_s^t) à fixer dès l'initialisation, nous cherchons à combiner la capacité d'apprentissage des GNN sur les données historiques avec la puissance de résolution exacte du solveur sur un espace de recherche drastiquement réduit. Néanmoins, l'enjeu majeur réside dans la préservation de la faisabilité. Une fixation trop rigide (par exemple un ratio de 0,95) peut restreindre l'espace de telle sorte qu'aucune solution ne respecte les contraintes budgétaires ou réglementaires.

5.2 Représentation des données (Entrées et Sorties)

La représentation des données s'inspire de [7], qui modélise le problème sous forme de graphe bipartite où les variables et les contraintes constituent les deux ensembles de nœuds, reliés dès qu'une variable apparaît dans une contrainte.Nous avons toutefois adapté cette structure aux spécificités de notre MILP. Puisque les variables de décision z_s^t permettent de déduire l'ensemble des autres variables, nous avons choisi de ne conserver que celles-ci dans le graphe.De plus, en exploitant la croissance des z_s^t, nous avons simplifié la représentation : chaque site est modélisé par un nœud unique dont l'objectif est de prédire l'instant de déploiement de la NG (ou -1 en l'absence de déploiement). Cette approche permet de s'affranchir des nœuds "contraintes" ; dans ce nouveau graphe, deux sites sont reliés par une arête s'ils couvrent une zone géographique commune.

Soit $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ le graphe représentant notre instance, où $\mathcal{V}$ est l'ensemble des sites candidats et $\mathcal{E}$ l'ensemble des arêtes.

- **Entrée ($\mathbf{X} \in \mathbb{R}^{N\times 2}$)** : La matrice des caractéristiques des nœuds, de dimension N (nombre de sites), définit chaque site par un vecteur incluant son degré (nombre de zones couvertes) et son potentiel client (demande totale couverte). Ces deux paramètres ont été sélectionnés comme étant les plus pertinents pour caractériser l'influence de chaque nœud dans le modèle.
- **Structure ($\mathbf{A} \in \mathbb{R}^{N\times N}$)** : La matrice d'adjacence du graphe définie par les arêtes $\mathcal{E}$.
- **Sortie ($\hat{\mathbf{Y}} \in \mathbb{R}^{N\times C}$)** : Matrice des probabilités (Logits) où $C = |T| + 1$ est le nombre de classes possibles (les différents pas de temps d'activation plus la classe "jamais activé").

5.3 Architecture du Graph Neural Network (MILPGNN)

La table 9 détaille les dimensions des tenseurs à travers les différentes couches du réseau, avec une dimension cachée paramétrée à $H = 64$.

5.4 Formulation Mathématique de la Couche SAGEConv

Le réseau GNN utilisé repose sur des couches de type **GraphSAGE** (SAmple and aggreGatE) proposées par Hamilton et al. (2017)[6]. Contrairement aux réseaux de neurones convolutifs classiques (CNN) qui opèrent sur des grilles fixes, GraphSAGE est conçu pour générer des plongements (*embeddings*) de nœuds en agrégeant les informations de leur voisinage local.

Nom de la couche	Dimension d'entrée	Dimension de sortie	Activation
1. SAGEConv_1	$N \times 2$	$N \times 64$	ReLU
2. SAGEConv_2	$N \times 64$	$N \times 64$	ReLU
3. SAGEConv_3	$N \times 64$	$N \times 64$	ReLU
4. Linear_1 (Classifier)	$N \times 64$	$N \times 32$	ReLU
5. Linear_2 (Classifier)	$N \times 32$	$N \times C$	- (CrossEntropy)

Table 9 – Architecture détaillée du MILPGNN. N correspond au nombre de sites et C au nombre d'instants t possibles plus la non-activation.

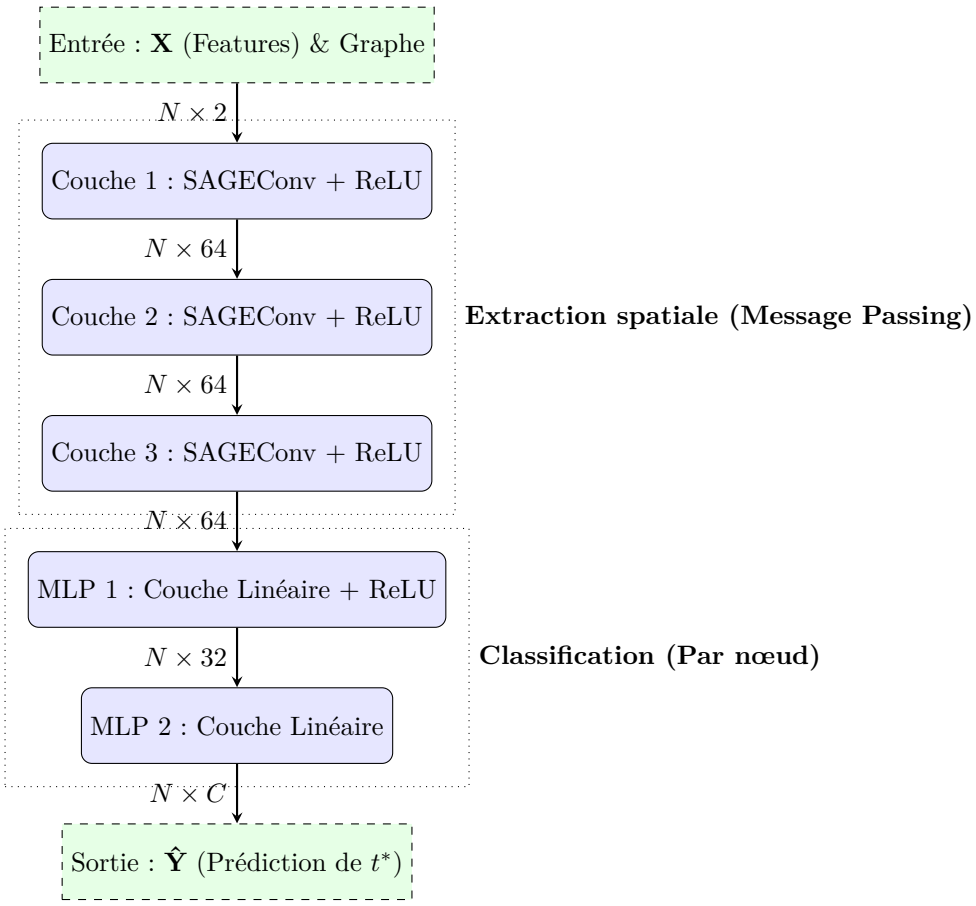


Figure 11 – Flux des données à travers les couches du GNN.

Soit un graphe $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. Pour un nœud $v \in \mathcal{V}$ (représentant un site candidat), on note $\mathcal{N}(v)$ l'ensemble de ses voisins directs. On définit $\mathbf{h}_v^{(l)}$ comme le vecteur de caractéristiques (*embedding*) du nœud v à la couche l.

À l'initialisation ($l = 0$), les caractéristiques du nœud correspondent aux données d'entrée brutes (le degré et la demande couverte potentielle) :

$$\mathbf{h}_v^{(0)} = \mathbf{x}_v \tag{9}$$

Pour chaque couche $l \in \{1, 2, 3\}$, la mise à jour de la représentation du nœud v s'effectue en deux étapes

mathématiques principales :

1. Étape d'agrégation (Message Passing) Le nœud v collecte et agrège les représentations de ses voisins de la couche précédente. La couche `SAGEConv` standard (telle qu'implémentée dans PyTorch Geometric) utilise l'opérateur de moyenne (*mean aggregator*) :

$$\mathbf{h}_{\mathcal{N}(v)}^{(l)} = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(l-1)} \tag{10}$$

Où $\mathbf{h}_{\mathcal{N}(v)}^{(l)}$ représente le message agrégé issu du voisinage immédiat du site v.

2. Étape de combinaison et d'activation Une fois le message du voisinage calculé, le nœud le combine avec sa propre représentation de la couche précédente. Une transformation affine est appliquée aux deux composantes via des matrices de poids apprenables $\mathbf{W}_1^{(l)}$ et $\mathbf{W}_2^{(l)}$, suivie d'une fonction d'activation non-linéaire σ (dans notre cas, la fonction ReLU) :

$$\mathbf{h}_v^{(l)} = \sigma\left(\mathbf{W}_1^{(l)} \mathbf{h}_v^{(l-1)} + \mathbf{W}_2^{(l)} \mathbf{h}_{\mathcal{N}(v)}^{(l)}\right) \tag{11}$$

En développant l'équation complète de la couche `SAGEConv`, la mise à jour s'écrit formellement :

$$\mathbf{h}_v^{(l)} = \text{ReLU}\left(\mathbf{W}_1^{(l)} \mathbf{h}_v^{(l-1)} + \mathbf{W}_2^{(l)} \cdot \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} \mathbf{h}_u^{(l-1)}\right) \tag{12}$$

Justfication du choix de 3 couches de type GraphSAGE :
Avec 3 couches de type GraphSAGE, chaque site reçoit des informations de ses voisins directs, mais aussi de ses "voisins de voisins" (distance de 2) et des "voisins de voisins de voisins" (distance de 3). Ainsi, chaque site reçoit une vue globale sur tous le graphe.

5.5 Résultats et analyse

L'un des grands avantages des GNN (et de SAGEConv) est leur capacité de généralisation inductive : vous pouvez entraîner sur de petites instances et inférer sur de grandes [6] [7].

Afin de garantir la pertinence des tests, les instances ont été choisies pour leur niveau de difficulté intermédiaire : suffisamment dimensionnées pour ne pas être triviales, mais restant traitables par le solveur dans un temps raisonnable. Notre choix s'est porté sur un échantillon de 100 instances de 50 zones générées aléatoirement.

Les performances du modèle ont été évaluées sur un ensemble de 10 instances, chacune comprenant 200 zones sélectionnées de manière aléatoire. Les tests ont été réalisés pour différentes valeurs du ratio, avec un temps de calcul maximal fixé à 30 minutes par exécution.

Comme attendu, l'augmentation du ratio de variables z_s^t fixées par le GNN entraîne une diminution conjointe de la valeur objectif et du temps de résolution. Les résultats obtenus avec un ratio de 0.5 sont particulièrement remarquables, offrant une solution très proche de l'optimum. Le passage à un ratio de 0.8 permet de réduire drastiquement le temps de calcul tout en maintenant une qualité de solution tout à fait acceptable. Cependant, une limite opérationnelle est franchie à 0.95, seuil à partir duquel le modèle commence à générer des solutions infaisables. En définitive, le paramétrage de ce ratio relèvera d'un arbitrage direct entre la précision recherchée et l'efficacité computationnelle.

Table 10 – Résultats des tests - Instances 0 à 4 (Temps | Valeur Objectif)

Modèle	**Seed 0**	**Seed 1**	**Seed 2**	**Seed 3**	**Seed 4**
MILP	$>$ 30 min —	405.1s 161574.08	439.3s 162189.90	647.4s 155399.60	129.9s 160054.83
MILP GNN (ratio=0.5)	420.5s 142981.94	54.2s 156063.63	186.8s 158860.52	176.6s 154200.97	118.3s 155706.68
MILP GNN (ratio=0.8)	38.2s 126771.94	38.6s 137349.95	41.0s 133756.94	47.4s 136770.21	39.8s 137543.52
MILP GNN (ratio=0.95)	Solution Infaisable	Solution Infaisable	Solution Infaisable	Solution Infaisable	Solution Infaisable

Table 11 – Résultats des tests - Instances 5 à 9 (Temps | Valeur Objectif)

Modèle	**Seed 5**	**Seed 6**	**Seed 7**	**Seed 8**	**Seed 9**
MILP	323.9s 156242.01	1385.9s 151160.24	61.1s 141164.56	625.1s 151396.95	716.8s 152941.26
MILP GNN (ratio=0.5)	131.3s 153463.55	78.7s 148508.27	91.1s 138145.64	51.5s 144816.85	107.7s 146068.14
MILP GNN (ratio=0.8)	42.4s 140008.34	57.3s 128334.89	43.2s 122574.09	37.5s 132725.20	52.0s 126783.81
MILP GNN (ratio=0.95)	30.8s 117077.02	Solution Infaisable	Solution Infaisable	Solution Infaisable	Solution Infaisable

6
APPRENTISSAGE PAR RENFORCEMENT

6.1 Motivations et positionnement

Dans le MILP, même si on modélise plusieurs périodes $t \in \mathcal{T}$, le solveur voit l'horizon entier d'un coup et optimise simultanément toutes les décisions z_s^t. En pratique, Orange ne connaît pas à l'avance avec certitude comment les abonnés vont migrer ni comment les concurrents vont évoluer. Le RL (Reinforcement Learning) modélise explicitement cette causalité temporelle : à chaque période t, l'agent observe ce qui s'est passé (combien d'abonnés ont migré, quelle est la couverture actuelle, etc) et adapte sa décision en conséquence. Il apprend non pas une solution figée, mais une règle de décision conditionnelle à l'état.

De plus, même si le MILP donne la solution optimale, son temps de calcul explose avec la taille de l'instance. En France, Orange a plus de 71 000 sites. Avec 5 périodes et 4 opérateurs, le nombre de variables et de contraintes devient prohibitif. Les heuristiques (FixRelax, génétique, mémétique) réduisent ce coût, mais à chaque nouvelle instance — nouvelle zone géographique, nouveaux paramètres de migration, nouveau budget — il faut relancer tout le processus.

Un agent PPO (Proximal Policy Optimization) entraîné une fois produit sa décision en quelques millisecondes, quelle que soit la taille de l'instance, simplement en évaluant le réseau de neurones sur l'état courant, car la politique apprise encode implicitement une logique de déploiement transférable.

6.2 Formulation MDP

Le problème de déploiement est naturellement formulé comme un MDP *(Markov Decision Process)* à horizon fini $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, dont les composantes sont définies ci-dessous.

Espace d'états $\mathcal{S}$. À chaque période $t \in \mathcal{T}$, l'état s_t encode l'ensemble des informations nécessaires à la décision courante :

$$s_t = \big(z_t,\ r_t,\ \mu_t,\ \mathcal{R}_t^{\text{comp}},\ \tau_t,\ b_t,\ \delta_t\big), \tag{13}$$

où $z_t \in \{0,1\}^{|\mathcal{S}|}$ est le vecteur de déploiement courant, $r_t \in \{0,1\}^{|A|}$ le vecteur de couverture ORANGE, $\mu_t \in \mathbb{R}^{|A|\times|I|\times|\mathcal{O}|}$ la distribution normalisée des abonnés, $\mathcal{R}_t^{\text{comp}}$ la couverture des concurrents à l'instant t, $\tau_t = t/T$ le temps normalisé, $b_t = Z^t/Z^{\max}$ le budget relatif disponible et $\delta_t = QA^t - \text{cov}(r_t)$ l'écart à la contrainte réglementaire.

Espace d'actions $\mathcal{A}$. À chaque période, l'agent produit un vecteur de scores $\phi \in \mathbb{R}^{|\mathcal{S}|}$, un score par site candidat. Les Z^t sites non encore déployés ayant les scores les plus élevés sont sélectionnés pour déploiement. Cette formulation *continue* de l'action, préférable à une sélection binaire directe, rend l'espace d'actions différentiable et compatible avec les algorithmes de gradient de politique.

Fonction de transition $\mathcal{P}$. La transition d'état est déterministe conditionnellement à l'action : les nouvelles couvertures r_{t+1} se déduisent de z_{t+1}, et les flux d'abonnés μ_{t+1} sont calculés par la fonction de migration `UPGRADE_FUNCTION`, éventuellement bruitée d'un terme gaussien de variance σ^2 pour modéliser l'incertitude.

Fonction de récompense $\mathcal{R}$. La récompense à chaque pas de temps est conçue pour aligner les incitations de l'agent avec les objectifs opérationnels et réglementaires de l'opérateur :

$$R_t = \underbrace{\alpha \cdot \Delta MS_t}_{\text{gain part de marché}} + \underbrace{\beta \cdot \Delta\text{Cov}_t}_{\text{gain couverture}} \underbrace{- \lambda_{\text{reg}} \cdot \max(0,\, QA^t - \text{cov}_t)^2}_{\text{pénalité réglementaire}} - \underbrace{\lambda_{\text{cap}} \cdot \text{Exc}_t}_{\text{pénalité capacité}} + \underbrace{\eta \cdot MS_T \cdot \mathbf{1}_{t=T}}_{\text{bonus terminal}}, \tag{14}$$

où ΔMS_t désigne l'incrément de part de marché 5G d'ORANGE entre $t-1$ et t, ΔCov_t l'incrément de couverture populationnelle, et Exc_t l'excès de trafic par rapport à la capacité agrégée des sites déployés. Le bonus terminal $\eta \cdot MS_T$ encourage l'agent à maximiser la part de marché finale, conformément à l'objectif du PLNE.

Les hyperparamètres de la récompense ont été fixés à $\alpha = 1.0$, $\beta = 0.3$, $\lambda_{\text{reg}} = 5.0$, $\lambda_{\text{cap}} = 2.0$ et $\eta = 2.0$.

6.3 Algorithme PPO et architecture

Choix de l'algorithme. Parmi les algorithmes de RL modernes, *Proximal Policy Optimization* (PPO) [8] a été retenu pour les raisons suivantes. Premièrement, PPO est un algorithme *on-policy* à gradient de politique, dont la mise à jour repose sur un ratio d'importance clippé :

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t\Big[\min\big(r_t(\theta)\,\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\,\hat{A}_t\big)\Big], \tag{15}$$

où $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ est le ratio de vraisemblance et $\hat{A}_t$ l'estimateur d'avantage (GAE, $\lambda = 0.95$). Le clipping empêche des mises à jour trop agressives de la politique, ce qui est crucial dans notre cadre où les épisodes sont courts ($T = 5$ pas) et où chaque déploiement est irréversible. Deuxièmement, PPO est simple à implémenter avec des environnements vectorisés, ce qui permet d'exploiter le parallélisme.

Architecture neuronale. La politique π_θ et la fonction de valeur V_θ partagent un encodeur MLP à trois couches denses $(256, 256, 128)$ avec activation ReLU, suivi de têtes séparées pour l'acteur et le critique. L'entrée est le vecteur d'état s_t de dimension :

$$d_{\text{obs}} = |\mathcal{S}| + |A| + |A| \cdot |I| \cdot |\mathcal{O}| + |A| \cdot N_{\text{comp}} + 3, \tag{16}$$

soit typiquement plusieurs centaines de dimensions selon la taille de l'instance. La sortie de l'acteur est un vecteur de scores $\phi \in \mathbb{R}^{|\mathcal{S}|}$ dans $[-10, 10]$.

Entraînement. L'entraînement utilise 8 environnements parallèles (`DummyVecEnv`) avec normalisation des observations et des récompenses (`VecNormalize`, $\gamma = 0.99$). Les hyperparamètres principaux sont récapitulés dans le tableau suivant.

Hyperparamètre	**Valeur**	**Rôle**
Taux d'apprentissage	3×10^{-4}	Pas de l'optimiseur Adam
Longueur de rollout	128 pas/env	Données collectées avant mise à jour
Taille de mini-batch	256	Taille des sous-lots pour le gradient
Époques par rollout	10	Nombre de passes sur les données
Facteur de discount γ	0.99	Pondération des récompenses futures
Clipping ε	0.2	Contrainte de mise à jour de la politique
Coefficient d'entropie	0.01	Bonus d'exploration
Timesteps totaux	500 000	Budget d'entraînement
Bruit de migration σ	0.05	Robustesse à l'incertitude

Table 12 – Hyperparamètres d'entraînement du modèle PPO.

Remarque sur la taille fixe de l'entrée. L'architecture MLP impose une dimension d_{obs} fixée à la compilation du réseau. Cela signifie que la politique entraînée sur une instance donnée (nombre de sites $|S|$ et de zones $|A|$ fixés) ne peut pas être directement transférée à une instance de dimension différente sans réentraînement. Ce point constitue une limite structurelle de l'approche MLP, et motive le recours à des architectures invariantes à la taille — telles que les réseaux de neurones sur graphe (GNN) présentés à la section 5.2 — comme perspective directe d'extension de ce travail.

6.4 Résultats et analyse

L'agent est évalué de manière déterministe ($\sigma = 0$) sur 10 épisodes complets et comparé à trois stratégies de référence : l'heuristique gloutonne (**Greedy**) (déployer les zones les plus peuplées), une politique aléatoire (**Random**).

Analyse du gain de part de marché. L'agent PPO atteint une part de marché finale de **65.3 %**, soit près du double de l'heuristique gloutonne (34.6 %). Ce résultat, à première vue surprenant, s'explique par la nature fondamentalement différente des deux stratégies.

L'heuristique gloutonne score chaque site par la *population géographiquement couverte* — c'est une bonne approximation de la couverture, mais elle ignore complètement qui sont les abonnés dans ces zones et vers où ils vont migrer. Un site couvrant une large population dont les abonnés sont déjà en 5G chez un concurrent apporte peu de valeur en termes de gain de part de marché.

Métrique	PPO	Greedy	Aléatoire
Récompense totale	**609 392**	322 669	454 415
Nombre d'abonnés 5G	**203 155**	107 644	151 511
Part de marché 5G finale	**65.3 %**	34.6 %	48.7 %
Couverture populationnelle	100 %	100 %	100 %
Sites déployés	50	50	50
Conformité réglementaire	60 %	**100 %**	**100 %**

Table 13 – Comparaison des stratégies sur l'instance Mayenne (moyenne sur 10 épisodes, évaluation déterministe).

L'agent PPO, en revanche, reçoit dans son état la distribution complète des abonnés μ_t par zone, par opérateur et par offre. Au fil de l'entraînement, il apprend à corréler ces informations avec les décisions qui maximisent la part de marché finale. Il découvre par exemple qu'il est plus profitable de déployer dans les zones où les abonnés 3G/4G des concurrents sont nombreux et susceptibles de migrer, plutôt que dans des zones déjà saturées ou peu dynamiques.

La non-conformité réglementaire : une limite cohérente. Cependant, cette performance doit être interprétée avec précaution. L'agent PPO viole la contrainte réglementaire QA dans 40 % des pas de temps, ce qui signifie que sa solution est infaisable au sens du MILP. En exploitant des régions de l'espace des solutions interdites par les contraintes réglementaires, PPO peut atteindre un nombre d'abonnés supérieur à celui du MILP (169 859 sur la même instance), qui lui garantit le respect strict de toutes les contraintes. Ce comportement n'est pas un dysfonctionnement de l'algorithme : c'est une conséquence naturelle de la structure de la récompense.

En multipliant le paramètre λ_{reg} par 2, ce qui revient à le fixer à 10, on obtient une valeur de la fonction objectif égale à 202 757, tout en respectant la contrainte réglementaire dans 80 % des périodes. Cela montre qu'un ajustement adéquat des paramètres de pénalisation permet de trouver un compromis satisfaisant entre l'optimisation de la fonction objectif et le respect des contraintes.

7 EXTENSION ÉNERGÉTIQUE DU PROBLÈME

7.1 Motivations et positionnement

Les formulations développées jusqu'ici supposent implicitement que l'énergie disponible sur chaque site est illimitée. Cette hypothèse, déjà fragile dans des contextes où les réseaux électriques sont stables, devient clairement intenable dans le contexte africain où une part significative des sites radio dépendent de sources contraintes (diesel, panneaux solaires, batteries) et où le déploiement de la 5G – intrinsèquement plus énergivore que les générations précédentes – se heurte à une limite physique avant même d'atteindre la limite budgétaire.

L'extension consiste dès lors à introduire un **second levier de décision** complémentaire au déploiement NG : le retrait progressif des couches legacy (3G, 4G) pour transférer leur enveloppe énergétique vers la NG. Le problème devient une optimisation *conjointe déploiement–retrait*, dans laquelle l'opérateur arbitre, site par site et période par période, entre trois actions : déployer la NG, maintenir une couche legacy, ou retirer une couche legacy. La 2G est conservée comme couche de service minimale non négociable – essentielle pour les services d'urgence et la voix bas débit – ce qui justifie l'absence de variable de décision associée.

7.2 Variables de décision et paramètres additionnels

Variables legacy. On introduit les binaires de maintien et de couverture legacy, pour $g \in \mathcal{G} = \{3G, 4G\}$. Pour toute zone $a \in \mathcal{A}$, on note $\mathcal{S}^g_{a,\tau}$ l'ensemble des sites de l'opérateur τ qui couvrent la zone a au moyen de la technologie legacy g. On définit alors :

$$x_s^{g,t} \in \{0,1\}, \quad r_a^{g,t} \in \{0,1\}, \qquad \forall\, s \in \mathcal{S}_\tau,\ a \in \mathcal{A},\ g \in \mathcal{G},\ t \in \mathcal{T}, \tag{17}$$

où $x_s^{g,t} = 1$ si la technologie g est maintenue sur le site s à la période t, et $r_a^{g,t} = 1$ si la zone a bénéficie encore d'une couverture par la technologie g.

Enfin, on pose $x_s^{g,0} = 1$ pour tout site initialement équipé de la technologie g, ce qui reflète fidèlement le parc existant.

Paramètres additionnels. L'extension requiert les paramètres résumés dans la table 14.

Paramètre	Signification
$E^{2G}_{s,t}$	Consommation fixe de la 2G sur le site s à la période t
$E^{g\text{-fix}}_{s,t}$, $g \in \mathcal{G}$	Consommation fixe de la couche legacy g sur le site s
$E^{5G\text{-fix}}_{s,t}$	Consommation fixe d'un site équipé NG
$E^{\max}_t$	Budget énergétique global de l'opérateur à la période t
δ_s	Facteur d'émission CO_2 par unité d'énergie du site s
$\mathrm{CO}^t_{2,\max}$	Plafond global d'émissions à la période t
$L^{\max}_g$	Seuil d'usage legacy résiduel autorisant le retrait
$\overline{Z}^t$	Budget global d'intervention réseau à la période t
$\eta^{g,i}_{a,C}$	Fraction des clients évincés de g-τ allant chez l'opérateur i sous le contexte C
M	Constante Big-M, majorant naturel du nombre de clients

Table 14 – Paramètres additionnels introduits par l'extension énergétique.

7.3 Modélisation de la consommation et de l'empreinte carbone

Principe général. La consommation d'une génération technologique g sur un site se décompose en principe en une part fixe, liée à l'activation des équipements, et une part variable, proportionnelle au trafic servi[15]. Dans la suite, nous **négligeons la part variable pour toutes les générations** : le socle fixe domine largement le profil de consommation d'une station radio, et cette simplification permet de conserver un modèle linéaire sans alourdir la formulation par une donnée additionnelle que nous n'avons pas calibrée. Chaque génération ne contribue alors que par son socle, conditionné à son état d'activation : la 2G est toujours active ; la legacy $g \in \mathcal{G}$ est active si $x_s^{g,t} = 1$; la NG est active si $z_s^t = 1$.

Consommation totale par site. La somme sur toutes les générations présentes donne :

$$E_{s,t} = E^{2G}_{s,t} + \sum_{g \in \mathcal{G}} x_s^{g,t}\, E^{g\text{-fix}}_{s,t} + E^{5G\text{-fix}}_{s,t}\, z_s^t, \qquad \forall\, s \in \mathcal{S}_\tau,\ t \in \mathcal{T}. \tag{18}$$

Budgets énergétique et carbone. L'opérateur dispose d'enveloppes totales par période :

$$\sum_{s \in \mathcal{S}_\tau} E_{s,t} \le E^{\max}_t, \qquad \sum_{s \in \mathcal{S}_\tau} \delta_s\, E_{s,t} \le \mathrm{CO}^t_{2,\max}, \qquad \forall\, t \in \mathcal{T}. \tag{19}$$

Ces deux contraintes ne sont pas redondantes dès que les δ_s sont hétérogènes : deux plans isoénergétiques peuvent produire des empreintes carbone très différentes selon la répartition spatiale. La formulation en contrainte plutôt qu'en pénalité dans l'objectif préserve l'interprétabilité des paramètres (kWh, tonnes de CO_2) et autorise une lecture économique directe des variables duales.

7.4 Règles et conditions du retrait legacy

Monotonie du retrait. Une technologie retirée ne peut être réactivée – la remobilisation matérielle et la renégociation des licences rendent ce scénario opérationnellement aberrant :

$$x_s^{g,t} \leq x_s^{g,t-1}, \qquad \forall\, s \in \mathcal{S}_\tau,\ g \in \mathcal{G},\ t \geq 1. \tag{20}$$

Couplage NG–retrait. Le retrait d'une legacy n'est autorisé que si la NG est déjà déployée sur le site. retirer la 3G/4G sur un site où notre opérateur n'a pas de NG revient à abandonner tous les clients legacy de ce site aux concurrents, sans aucune contrepartie pour lui :

$$z_s^t \geq 1 - x_s^{g,t}, \qquad \forall\, s \in \mathcal{S}_\tau,\ g \in \mathcal{G},\ t \in \mathcal{T}. \tag{21}$$

Seuil d'usage commercial. Le retrait n'est autorisé que si le trafic résiduel sur la couche legacy est sous un seuil $L_g^{\max}$ d'acceptabilité commerciale (typiquement quelques pourcents de la population initialement abonnée) :

$$\sum_{a \in \mathcal{A}_s} u^t_{a,\tau,o_\tau^g} \leq L_g^{\max} + M\, x_s^{g,t}, \qquad \forall\, s \in \mathcal{S}_\tau,\ g \in \mathcal{G},\ t \in \mathcal{T}. \tag{22}$$

Couverture legacy. Les variables $r_a^{g,t}$ se couplent à x par un double jeu d'inégalités :

$$r_a^{g,t} \leq \sum_{s \in \mathcal{S}^g_{a,\tau}} x_s^{g,t}, \qquad x_s^{g,t} \leq r_a^{g,t}\ \ (\forall s \in \mathcal{S}^g_{a,\tau}), \qquad \forall\, a \in \mathcal{A},\ g \in \mathcal{G},\ t \in \mathcal{T}. \tag{23}$$

Combinées, ces deux conditions forcent $r_a^{g,t} = 1$ si et seulement si au moins un site couvrant a maintient la technologie g.

7.5 Dynamique migratoire avec retrait et redistribution

La fonction $f^a_{C,o',o}$ est une donnée exogène issue des fichiers terrain : elle décrit la dynamique migratoire en fonction de la couverture NG **mais ne sait pas qu'une couche legacy est retirée**. En particulier, le coefficient $f^a_{C,o_\tau^g,o_\tau^g}$ –qui modélise les clients restant dans leur offre legacy d'une période à l'autre – continuerait à maintenir des clients sur une offre qui n'existe plus, créant une incohérence entre la population comptabilisée et la couverture effective. Nous dédoublons la variable de population en deux niveaux pour résoudre cette incohérence à la source, sans modification de f.

Population naturelle. La variable auxiliaire $\tilde{u}^t_{a,i,o}$ représente la population prédite par f *comme si aucune legacy n'était retirée*. Elle est gouvernée par l'équation migratoire suivante :

$$\tilde{u}^t_{a,i,o} = \sum_{C \in \{0,1\}^{|I|}} \delta^t_{a,C} \sum_{i' \in I} \sum_{o' \in \mathcal{O}_{i'} \cup \{NO_{i'}\}} f^a_{C,o',o}\, u^{t-1}_{a,i',o'}, \qquad \forall\, a, i, o, t. \tag{24}$$

Correction pour les offres legacy de τ.

$$u^t_{a,\tau,o_\tau^g} = \tilde{u}^t_{a,\tau,o_\tau^g} \cdot r_a^{g,t}, \qquad \forall\, a \in \mathcal{A},\ g \in \mathcal{G},\ t \in \mathcal{T}. \tag{25}$$

Si $r_a^{g,t} = 1$ (couverture maintenue), alors $u = \tilde{u}$ et le comportement est identique à celui du modèle de base. Si $r_a^{g,t} = 0$ (retrait), alors $u = 0$: éviction stricte des clients de l'offre disparue.

Redistribution des clients évincés. La modélisation des clients évincés par la suppression d'une couche legacy repose sur l'hypothèse qu'un client évincé se comporte en moyenne comme un client qui aurait migré spontanément, et se répartit entre opérateurs selon les *mêmes* proportions relatives que celles observées dans f. Cette démarche, compatible avec l'Hypothèse 2 du modèle de base (migrations uniquement ascendantes) et ne nécessitant aucune donnée supplémentaire, est formalisée par le coefficient de redistribution défini pour chaque zone a, chaque legacy $g \in \mathcal{G}$, chaque opérateur destinataire $i \in I$ et chaque vecteur de couverture C par :

$$\eta^{g,i}_{a,C} = \frac{f^{a}_{C,\, o^{g}_{\tau},\, NO_i}}{\displaystyle\sum_{j \in I} f^{a}_{C,\, o^{g}_{\tau},\, NO_j}}, \qquad \text{avec } \sum_{i \in I} \eta^{g,i}_{a,C} = 1. \tag{26}$$

Ces coefficients sont des *paramètres précalculés* à partir des fichiers terrain ; ils ne rajoutent aucune variable au modèle. Ils sont bien définis, puisqu'il existe toujours au moins une migration possible pour chaque zone. Les clients évincés sont alors répartis proportionnellement à ces préférences, pour tout opérateur $i \in I$:

$$u^{t}_{a,i,NO_i} = \tilde{u}^{t}_{a,i,NO_i} + \sum_{g \in \mathcal{G}} \eta^{g,i}_{a,\, C^{t}_{a}} \left(1 - r^{g,t}_{a}\right) \tilde{u}^{t}_{a,\tau,o^{g}_{\tau}}, \qquad \forall\, a \in \mathcal{A},\ i \in I,\ t \in \mathcal{T}. \tag{27}$$

Pour les autres combinaisons (i, o) non concernées par l'éviction, on pose $u^{t}_{a,i,o} = \tilde{u}^{t}_{a,i,o}$.

Conservation structurelle. En sommant $u^{t}_{a,i,o}$ sur toutes les offres et opérateurs, les termes de redistribution se recombinent exactement grâce aux propriétés $\sum_i \eta^{g,i}_{a,C} = 1$ (normalisation de la redistribution) et $\sum_{i,o} f^{a}_{C,o',o} = 1$ (conservation de f), ce qui donne :

$$\sum_{i,o} u^{t}_{a,i,o} = \sum_{i,o} u^{t-1}_{a,i,o}.$$

L'Hypothèse 1 du modèle de base – population constante par zone – est ainsi préservée *structurellement*, indépendamment des valeurs numériques de f et $r^{g,t}_{a}$.

7.6 Budget global d'intervention

La contrainte budgétaire du modèle initial limitait uniquement les *déploiements* NG. Un retrait legacy constituant également une intervention technique (visite de site, reconfiguration, démontage), elle est remplacée par un comptage global des opérations réseau :

$$\sum_{s \in \mathcal{S}_\tau} (z^{t}_{s} - z^{t-1}_{s}) + \sum_{s \in \mathcal{S}_\tau} \sum_{g \in \mathcal{G}} (x^{g,t-1}_{s} - x^{g,t}_{s}) \le \overline{Z}^{\,t}, \qquad \forall\, t \ge 1. \tag{28}$$

7.7 Modèle d'optimisation conjointe déploiement–retrait ($\mathcal{P}_E$)

Nous pouvons à présent formuler le problème d'optimisation conjointe déploiement–retrait. Celui-ci vise à déterminer, sur l'horizon de planification considéré, les décisions de déploiement de la NG, de maintien ou de retrait des technologies legacy, ainsi que les populations résultantes, de manière à maximiser la part de marché NG de l'opérateur cible sous contraintes de couverture, de capacité, d'énergie, de carbone et de budget d'intervention.

$$\max_{z,\,x,\,u,\,\tilde{u},\,r,\,r^g} \quad \sum_{a\in\mathcal{A}} u_{a,\tau,NO_\tau}^{t_{|\mathcal{T}|}} \tag{4.1}$$

s.t.

$$r_a^t \le \sum_{s\in\mathcal{S}_{a,\tau}} z_s^t \qquad \forall\, a\in\mathcal{A},\ t\in\mathcal{T}, \tag{4.2}$$

$$z_s^t \le r_a^t \qquad \forall\, a\in\mathcal{A},\ s\in\mathcal{S}_{a,\tau},\ t\in\mathcal{T}, \tag{4.3}$$

$$\tilde{u}_{a,i,o}^t = r_a^t \sum_{i',o'} f^a_{\langle 1,R_a^t\rangle,o',o}\, u_{a,i',o'}^{t-1} + (1-r_a^t)\sum_{i',o'} f^a_{\langle 0,R_a^t\rangle,o',o}\, u_{a,i',o'}^{t-1} \qquad \forall\, a\in\mathcal{A},\ i\in I,\ o\in\mathcal{O}_i\cup\{NO_i\},\ t\in\mathcal{T}, \tag{4.4}$$

$$u_{a,\tau,o_\tau^g}^t = r_a^{g,t}\cdot\tilde{u}_{a,\tau,o_\tau^g}^t \qquad \forall\, a\in\mathcal{A},\ g\in\mathcal{G},\ t\in\mathcal{T}, \tag{4.5}$$

$$u_{a,i,NO_i}^t = \tilde{u}_{a,i,NO_i}^t + \sum_{g\in\mathcal{G}} \eta^{g,i}_{a,\,C_a^t}\,(1-r_a^{g,t})\,\tilde{u}_{a,\tau,o_\tau^g}^t \qquad \forall\, a\in\mathcal{A},\ i\in I,\ t\in\mathcal{T}, \tag{4.6}$$

$$u_{a,i,o}^t = \tilde{u}_{a,i,o}^t \qquad \forall\,(i,o)\notin\{(\tau,o_\tau^g),\,(\cdot,NO_\cdot)\}, \tag{4.7}$$

$$u_{a,\tau,NO_\tau}^t = \sum_{s\in\mathcal{S}_{a,\tau}} u_{a,\tau,NO_\tau,s}^t \qquad \forall\, a\in\mathcal{A},\ t\in\mathcal{T}, \tag{4.8}$$

$$\sum_{a\in\mathcal{A}_s} D_{NG}^t\, u_{a,\tau,NO_\tau,s}^t \le \mathrm{CAPA}_{NG}^t\, z_s^t \qquad \forall\, s\in\mathcal{S}_\tau,\ t\in\mathcal{T}, \tag{4.9}$$

$$\sum_{s\in\mathcal{S}_\tau}(z_s^t - z_s^{t-1}) + \sum_{s\in\mathcal{S}_\tau}\sum_{g\in\mathcal{G}}(x_s^{g,t-1} - x_s^{g,t}) \le \overline{Z}^{\,t} \qquad \forall\, t\in\mathcal{T}\setminus\{0\}, \tag{4.10}$$

$$\sum_{a\in\mathcal{A}} u_a\, r_a^t \ge QA^t \sum_{a\in\mathcal{A}} u_a \qquad \forall\, t\in\mathcal{T}, \tag{4.11}$$

$$z_s^t \ge z_s^{t-1} \qquad \forall\, s\in\mathcal{S}_\tau,\ t\in\mathcal{T}\setminus\{0\}, \tag{4.12}$$

$$x_s^{g,t} \le x_s^{g,t-1} \qquad \forall\, s\in\mathcal{S}_\tau,\ g\in\mathcal{G},\ t\in\mathcal{T}\setminus\{0\}, \tag{4.13}$$

$$E_{s,t} = E_{s,t}^{2G} + \sum_{g\in\mathcal{G}} x_s^{g,t}\, E_{s,t}^{g\text{-fix}} + E_{s,t}^{5G\text{-fix}}\, z_s^t \qquad \forall\, s\in\mathcal{S}_\tau,\ t\in\mathcal{T}, \tag{4.14}$$

$$\sum_{s\in\mathcal{S}_\tau} E_{s,t} \le E_t^{\max} \qquad \forall\, t\in\mathcal{T}, \tag{3.15}$$

$$\sum_{s\in\mathcal{S}_\tau} \delta_s\, E_{s,t} \le \mathrm{CO}_{2,\max}^t \qquad \forall\, t\in\mathcal{T}, \tag{4.16}$$

$$z_s^t + x_s^{g,t} \ge 1 \qquad \forall\, s\in\mathcal{S}_\tau,\ g\in\mathcal{G},\ t\in\mathcal{T}, \tag{4.17}$$

$$\sum_{a\in\mathcal{A}_s} u_{a,\tau,o_\tau^g}^t \le L_g^{\max} + M\, x_s^{g,t} \qquad \forall\, s\in\mathcal{S}_\tau,\ g\in\mathcal{G},\ t\in\mathcal{T}, \tag{4.18}$$

$$r_a^{g,t} \le \sum_{s\in\mathcal{S}^g_{a,\tau}} x_s^{g,t} \qquad \forall\, a\in\mathcal{A},\ g\in\mathcal{G},\ t\in\mathcal{T}, \tag{4.19}$$

$$x_s^{g,t} \le r_a^{g,t} \qquad \forall\, a\in\mathcal{A},\ s\in\mathcal{S}^g_{a,\tau},\ g\in\mathcal{G},\ t\in\mathcal{T}, \tag{4.20}$$

$$r_a^t,\, r_a^{g,t},\, z_s^t,\, x_s^{g,t} \in \{0,1\} \qquad \forall\, a\in\mathcal{A},\ g\in\mathcal{G},\ s\in\mathcal{S}_\tau,\ t\in\mathcal{T}, \tag{4.21}$$

$$u_{a,i,o}^t,\, \tilde{u}_{a,i,o}^t \in \mathbb{R}_+ \qquad \forall\, a\in\mathcal{A},\ i\in I,\ o\in\mathcal{O}_i\cup\{NO_i\},\ t\in\mathcal{T}, \tag{4.22}$$

$$u_{a,\tau,NO_\tau,s}^t \in \mathbb{R}_+ \qquad \forall\, a\in\mathcal{A},\ s\in\mathcal{S}_{a,\tau},\ t\in\mathcal{T}, \tag{4.23}$$

$$E_{s,t} \in \mathbb{R}_+ \qquad \forall\, s\in\mathcal{S}_\tau,\ t\in\mathcal{T}. \tag{4.24}$$

8 CONCLUSION

Ce travail a abordé le problème d'optimisation du déploiement des technologies mobiles de nouvelle génération dans un contexte concurrentiel, en intégrant des contraintes de budget, de capacité et de couverture. À partir d'un modèle initial, une reformulation a permis de réduire le nombre de contraintes de 76 % (passant de 1 653 574 à 399 580) et le nombre de variables de 17 % en moyenne. Cette approche s'est montrée particulièrement efficace sur diverses instances, le nouveau modèle MILP réussissant à diminuer le temps de calcul moyen de 63 % (passant de 265,0 s à 96,9 s) tout en conservant une valeur objective quasi identique.

Cependant, les méthodes exactes restent coûteuses pour la résolution de grandes instances avec des données réelles. Le Fix-and-Relax s'est révélé inefficace dans ce contexte, avec des temps de calcul dépassant souvent 30 minutes sans aboutir à une solution réalisable.

Les alternatives heuristiques ont offert de meilleurs compromis :

- L'Algorithme Génétique (AG) direct a permis de réduire le temps d'exécution en moyenne de 87,7 % (pour les instances de 550 zones) et s'approche de la valeur optimale à hauteur de 81,6 %, tout en conservant une complexité polynomiale.
- L'Algorithme Mémétique, grâce à sa recherche locale, et avec une complexité polynomiale surpasse la version direct en atteignant en moyenne 97,8 % de la valeur optimale pour un temps de calcul moyen de 469,6 s sur de grandes instances (550 zones), soit une réduction de temps de 83 % par rapport au solveur.
- Le MILPGNN avec un ratio de 0,5 s'approche de la valeur optimale à 97,5 % avec une diminution du temps moyen de 71,6 %. Avec un ratio de 0,8, le temps de calcul chute de 91,7 % en moyenne. Toutefois, plus le ratio de variables prédites augmente, plus le gain de temps est important, mais plus le risque d'infaisabilité s'accroît, devenant critique à partir d'un ratio de 0,95. Ce risque d'infaisabilité est particulièrement marqué sur les grandes instances où la structure des contraintes est plus rigide [14] .
- Finalement, l'Apprentissage par Renforcement (PPO) offre une part de marché finale de 65,3 %, soit près du double de l'heuristique gloutonne (34,6 %). Son principal défaut réside dans sa difficulté à respecter strictement les contraintes, les solutions étant non conformes réglementairement dans 40 % des cas. De plus, l'architecture MLP utilisée nécessite un entraînement et un test sur des instances de même dimension, limitant sa flexibilité opérationnelle.

Une perspective directe serait d'approfondir l'extension énergétique proposée, en suivant la même démarche que pour les modèles précédents : linéariser les contraintes, l'implémenter, puis l'alimenter avec des données adaptées. Cette étape permettrait de comparer le modèle énergétique avec le modèle initial et d'évaluer l'effet réel de la prise en compte de l'énergie sur les décisions de déploiement.

Une autre perspective concerne l'amélioration des méthodes de résolution, en combinant les heuristiques développées avec des approches d'apprentissage automatique. En particulier, les Graph Neural Networks pourraient être utilisés pour mieux exploiter la structure du réseau et guider les heuristiques vers les sites les plus prometteurs, afin d'obtenir plus rapidement des solutions de bonne qualité sur de grandes instances. Les perspectives liées à l'apprentissage par renforcement, détaillées dans la section correspondante, ouvrent également une piste intéressante pour la suite, notamment pour mieux exploiter la dimension séquentielle du problème et évaluer le potentiel de PPO dans la prise de décision de déploiement.

Et pour aller plus loin on peut intégrer l'incertitude dans le modèle. En pratique, la demande future, les comportements de migration des clients et les stratégies des concurrents ne sont pas parfaitement connus. Il serait donc pertinent d'étendre le modèle vers une approche stochastique, capable de produire des stratégies efficaces même si les paramètres varient.

ANNEXE : MAPPING DES DONNÉES ET ALIMENTATION DU MODÈLE AVEC LES DONNÉES DU TERRAIN

Origine et description des données

Les données utilisées pour alimenter le modèle proviennent de l'instance Mayenne (723 sites et 616 zones) fournie par nos tuteurs d'Orange. Il s'agit d'un ensemble de fichiers décrivant notamment les zones géographiques, les sites existants, la demande moyenne, la capacité projetée en 5G, les contraintes stratégiques de déploiement, ainsi que la couverture concurrente et les fonctions de migration des clients entre les différentes offres. Ces éléments sont détaillés ci-dessous. Les fichiers utilisés sont les suivants :

- `AREAS.csv` : décrit pour chaque zone le nombre d'utilisateurs par technologie et par opérateur.
- `EXISTING_SITES.csv` : liste les sites existants de l'opérateur étudié (ici ORANGE), ainsi que l'état de déploiement de chaque site par la 3G, 4G et 5G.
- `AREAS_SITES_LINK.csv` : lie les zones aux sites via les colonnes `AREAS` et `SITES` (quels sites couvrent quelles zones).
- `COMPETITORS_STRATEGY.csv` : donne la présence ou non de couverture 5G des concurrents par période et par zone, ce qui représente la stratégie des concurrents.
- `DEMAND.csv` : fournit, par période, un coefficient de demande moyenne de trafic pour la 5G.
- `CAPACITY.csv` : donne la capacité maximale de trafic 5G supportable par un site par période.
- `OPERATIONAL_LIMITS.csv` : donne le nombre maximal de sites 5G pouvant être déployés à chaque période.
- `STRATEGIC_GUIDELINES.csv` : définit, par période, une cible de couverture minimale de la population par la 5G.
- `UPGRADE_FUNCTION.csv` : décrit les comportements de migration entre offres et technologies, sous forme de pourcentages de clients qui basculent vers une offre 5G, en fonction d'un contexte de couverture des opérateurs.

Ces fichiers alimentent directement la définition des **ensembles** et des **paramètres** du modèle.

Correspondance entre les fichiers et les paramètres du modèle

Le tableau suivant récapitule la correspondance entre chaque fichier source et les éléments du modèle mathématique qu'il alimente.

Table 15 – Correspondance fichiers sources – paramètres du modèle

Fichier source	Paramètre(s) alimenté(s)	Description
`AREAS.csv`	$u^0_{a,i,o}$, u_a	Population initiale par zone, opérateur et offre
`EXISTING_SITES.csv`	$\mathcal{S}_\tau$	Ensemble des sites de l'opérateur cible
`AREAS_SITES_LINK.csv`	$\mathcal{S}_{a,\tau}$, $\mathcal{A}_s$	Lien zones–sites (couverture géographique)
`COMPETITORS_STRATEGY.csv`	$R^t_{a,i}$	Couverture NG des concurrents par zone et période
`DEMAND.csv`	D^t_{NG}	Demande de trafic 5G par période
`CAPACITY.csv`	CAPA^t_{NG}	Capacité d'un site NG par période
`OPERATIONAL_LIMITS.csv`	$\overline{Z}^{\,t}$	Budget de déploiement par période
`STRATEGIC_GUIDELINES.csv`	$\underline{QA^t}$	Cible réglementaire de couverture minimale
`UPGRADE_FUNCTION.csv`	$f^a_{C,o',o}$	Fonction de migration entre offres

Un exemple précis : construction de la fonction de transition

Le fichier `UPGRADE_FUNCTION.csv` constitue l'entrée la plus structurée du modèle. Nous en détaillons ici le traitement.

Structure du fichier. Chaque ligne du fichier contient :

- un *pattern* de couverture codé par quatre colonnes binaires : `ORANGE`, `FREE MOBILE`, `BOUYGUES TELECOM`, `SFR`, définissant un vecteur
$$C^{csv} = (c_O, c_F, c_B, c_S) \in \{0,1\}^4 ;$$
- une offre de départ `OFFERS` associée à un `FROM_OPERATOR`, concaténées pour former
$$O_1 = \texttt{OFFERS} + \texttt{"-"} + \texttt{FROM_OPERATOR} ;$$
- un opérateur cible `TO_OPERATOR`, à partir duquel on construit l'offre 5G d'arrivée
$$O_2 = \texttt{"o5G-"} + \texttt{TO_OPERATOR} ;$$
- un pourcentage `PERCENTAGES`, représentant la proportion de clients passant de O_1 à O_2 sous le contexte de couverture C^{csv}.

Construction du paramètre f. À partir de ce fichier, on construit le paramètre f^a_{C,O_1,O_2} selon la procédure suivante.

Pour chaque ligne du CSV et pour le pattern C^{csv} correspondant, on fixe :
$$f^a_{C^{csv},O_1,O_2} = \texttt{PERCENTAGES} .$$
Les données ne variant pas par zone, la même matrice de pourcentages est répliquée pour toutes les zones $a \in \mathcal{A}$.

Pour s'assurer que la fonction est définie sur toutes les combinaisons (a, C, O_1, O_2), on complète systématiquement par :
$$f^a_{C,O_1,O_2} = 0$$
si aucune ligne de `UPGRADE_FUNCTION.csv` ne correspond au triplet (C, O_1, O_2).

Ainsi, f encode, pour chaque zone, chaque contexte de couverture C et chaque paire d'offres (O_1, O_2), la part de clients migrant de O_1 vers O_2.

RÉFÉRENCES

[1] Raúl Katz - Fernando Callorda - Ramiro Valencia : The independent tower industry as a key enabler of the developement of african telecominucatiosn.

[2] A. Cambier, M. Chardy, R. Figueiredo, A. Ouorou, and M. Poss : Optimizing the investments in mobile networks and subscriber migrations for a telecommunication operator. Networks, 77(4) :495–519, 2021.

[3] M. Chardy, M. Ben Yahia, and Y. Bao : 3g/4g load-balancing optimization for mobile network planning. pages 7–12, 09 2016

[4] Amal Benhamich, Matthieu Chardy, Brahim Mebrek : Modelling the mobile investment strategies under competition using mathematical programming.

[5] ANFR, *Au 1er avril 2026, plus de 54 600 sites 5G et 71 750 sites 4G autorisés par l'ANFR en France*

[6] W. Hamilton, R. Ying, et J. Leskovec : Inductive Representation Learning on Large Graphs, 2017

[7] Maxime Gasse, Didier Chételat, Nicola Ferroni, Laurent Charlin, et Andrea Lodi : Exact Combinatorial Optimization with Graph Convolutional Neural Networks, 2019.

[8] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov OpenAI : Proximal Policy Optimization Algorithms (2017)

[9] J. H. Holland, *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.

[10] Paolo Zappalà, *Méthodes de résolution des jeux en forme extensive avec application au marché des réseaux mobiles*, Thèse de doctorat, Avignon Université, 2024.

[11] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.

[12] P. Moscato, "On Evolution, Search, Optimization, Genetic Algorithms and Martial Arts : Towards Memetic Algorithms," Caltech Concurrent Computation Program, C3P Report, 1989.

[13] F. Neri and C. Cotta, "Memetic algorithms and memetic computing optimization : A literature review," *Swarm and Evolutionary Computation*, vol. 2, pp. 1–14, 2012.

[14] Jian-Ya Ding, Chao Zhang, Linxin Shen, Shengyu Li, Bing Wang, Yinghui Xu et Le Song, *Accelerating Primal Solution Findings for Mixed Integer Programs Based on Solution Prediction*

[15] Madhu Sudan Dahal, Jagan Nath Shrestha, Shree Raj Shakya, *Energy saving technique and measurement in green wireless communication*, Energy, vol. 159, pp. 21–31, 2018.